

Understanding simpleKafka from Scratch

A source-driven, incremental Java course

simpleKafka teaching project

Java 21 · 31 executable steps

Before you begin

This book develops the course through one map chapter and eight implementation chapters, covering 31 steps. You will build record formats, logs, network requests, consumer offsets, consumer groups, controlled replica failover, and consistency experiments across producers, brokers, and consumers. This is neither “implementing Kafka” by connecting to a real Kafka cluster nor reducing production Kafka to a few empty terms. Every step maps to an explicit Java method, behavior tests, failure boundaries, and a reproducible command. Start at Step 1 and proceed in order; the book, course manifest, and test classes share the same numbering.

The course deliberately separates Java sources into three roots: **provided/**, **exercises/**, and **reference/**. The student entry point compiles the first two; the reference entry point compiles the first and third. The two implementations have identical fully qualified class names, so never compile both answer roots together. The reference implementation is for comparison with the contract after you try a step, not a back door onto the student test classpath. Chapter 1 explains the development environment, how to read test failures, and the progression rules.

Contents

1	Course Map, Tools, and How to Learn	1
1.1	Scope of the Course Deliverable	1
1.2	Preparing Java 21+ and Course Commands	1
1.3	Understanding Binary Data and File I/O	2
1.4	How to Read Test Failures	2
1.5	Start with System Invariants	3
2	Records and Single-Segment Persistence	4
2.1	Step 1: Record Encoding and Decoding	4
2.2	Step 2: Log Append	6
2.3	Step 3: Offset-Based Reads	7
2.4	Step 4: Crash Recovery	8
3	Segments, Indexing, Retention, and Truncation	10
3.1	Step 5: Sparse Index	10
3.2	Step 6: Segmented Log	12
3.3	Step 7: Log Retention	13
3.4	Step 8: Tail Truncation	14
4	Partitions and a Broker on a Real Network	16
4.1	Step 9: Partition Routing	16
4.2	Step 10: Topic Metadata	17
4.3	Step 11: TCP Frame Protocol	19
4.4	Step 12: Single-Broker Requests	20
5	Producing, Consuming, Offsets, and Redelivery	22
5.1	Step 13: Batched Producer	22
5.2	Step 14: Pull-Based Consumer	23
5.3	Step 15: Consumer Offset Persistence	25
5.4	Step 16: At-Least-Once Processing	26
6	Consumer Groups, Rebalancing, and Commit Fencing	28
6.1	Step 17: Partition Assignment	28
6.2	Step 18: Membership and Rebalancing	29

6.3	Step 19: Heartbeats and Expiration	31
6.4	Step 20: Consumer Group Fencing	32
7	Replication, ISR, and High Watermarks	34
7.1	Step 21: Fetch-Based Replication	34
7.2	Step 22: ISR and High Watermark	36
7.3	Step 23: Produce Acknowledgments	37
7.4	Step 24: Concurrent Replicated Broker	39
8	Election, Log Repair, and Replica Readmission	41
8.1	Step 25: Clean Election and Leader Epochs	41
8.2	Step 26: Divergent-Log Reconciliation	43
8.3	Step 27: Metadata Routing Refresh	44
8.4	Step 28: Replica Readmission to ISR	46
9	Consistency Across Producers, Brokers, and Consumers	48
9.1	Step 29: Cross-Layer Consistency Experiments	48
9.2	Step 30: Idempotent Producer	50
9.3	Step 31: Idempotent Consumer Effects	53
A	Formats, Error Codes, and Interface Index	56
A.1	Disk and Network Formats	56
A.1.1	Records and Indexes	56
A.1.2	TCP Frames and Request Bodies	57
A.1.3	Message Body Field Order	57
A.1.4	Identifiers and Value Types	58
A.2	Idempotent Append and Durable Effects	59
A.3	Fixed Error Codes	59
A.4	The 31 Steps and Edit Entry Points	60
A.5	Course Simplifications and Differences from Kafka	61
A.6	Official References and Further Reading	62
A.7	Typesetting and Build Instructions	62
	Glossary	64

Chapter 1

Course Map, Tools, and How to Learn

1.1 Scope of the Course Deliverable

This project builds the main path through a messaging system as a small, runnable system: partition logs on disk, TCP requests, producing and consuming, consumer offsets, a single-topic consumer group, and pull-based replication with controlled failover among multiple local brokers. The subject is not the implementation of the official Kafka clients, nor the Kafka wire protocol. The binary protocol provided here serves this course only; do not point the official `KafkaProducer` or `KafkaConsumer` at this program.

The three Java source roots are part of the course design. `provided/` contains data types, I/O, transport, serialization scaffolding, lifecycle management, and test fixtures; it keeps the focus on algorithms rather than rebuilding boilerplate. `exercises/` is the only workspace students need to complete step by step. The types and public signatures for all steps are visible in advance, while methods not yet reached throw `ExerciseNotImplementedException` with the step number. `reference/` contains the isolated complete solution for comparison and course verification. Its class names match those in the exercise sources, so student and reference source roots must never be compiled together. Tests reuse the same black-box contracts in both modes; the student runtime classpath does not contain reference classes.

This is not production-grade Kafka: it has no Kafka wire-protocol compatibility, transactions, compression, compaction, complete controller consensus, or controller HA. Chapter 8 teaches a limited idempotent-producer protocol and a durable local effect ledger, but neither is a Kafka production protocol nor an end-to-end transaction. Its authority is a trusted teaching control plane within one JVM; although three brokers each have a TCP server, threads, and a disk directory, their replicas still share this authority. It can demonstrate controlled failover, but cannot be used to claim consensus under arbitrary network partitions or production-grade availability. The appendices and relevant steps repeatedly call out the specific differences.

1.2 Preparing Java 21+ and Course Commands

The repository uses the Gradle Wrapper `./gradlew`; Gradle 9.8.1 runs on Java 17+, while Java compile, test, and demo tasks require a Java 21+ JDK and compile with `--release21`. Maven, Docker, and a running Kafka are not required. The scripts use Bash 3.2+, `find`, `awk`, `curl`, and either `sha256sum` or `shasum`; `tar` and `install` prepare Tectonic, while macOS Pandoc extraction also needs `unzip`. EPUB diagrams require Poppler's `pdftoppm`; `./gradlew :setupBook` checks for it but does not install system packages. Install Poppler on Linux/WSL with `sudo`

`apt-get install poppler-utils`, or on macOS with `brew install poppler`. First check `java -version`, `javac -version`, and `./gradlew --version`. You may use any suitable JDK through `JAVA_HOME` or `PATH`; `SDKMAN` is optional. The project does not silently change `SDKMAN`, system packages, or shell configuration.

`./gradlew :setup` resolves the pinned JUnit 1.11.4 dependency and verifies its checksum through Gradle; it does not store JUnit under `.tools/`. `./gradlew :setupBook` prepares checksum-verified Tectonic 0.17.0 and Pandoc 3.12.1 under `.tools/`, and checks Poppler without installing system software. `./gradlew :doctor` checks Gradle, Java, managed JUnit, and optional book tools. `./gradlew :compile` compiles the student skeleton and tests; successful compilation does not mean the exercises are implemented. The default course and list language is English; `COURSE_LANG=zh ./gradlew :list` displays Chinese step titles. `./gradlew :book` builds the English PDF and EPUB at `build/book/en/simpleKafka.pdf` and `build/book/en/simpleKafka.epub`; `COURSE_LANG=zh ./gradlew :book` builds the Chinese pair at `build/book/zh/simpleKafka-zh.pdf` and `build/book/zh/simpleKafka-zh.epub`. Each command also updates that language's checked-in PDF and EPUB under `docs/book/dist/`. Readers can open those distributed files without Java or build tools. The book task starts the Wrapper JVM but does not compile Java or resolve JUnit. The first build needs a network connection for the TeX bundle; after warming both languages, use `BOOK_OFFLINE=1 ./gradlew :book --offline` and `COURSE_LANG=zh BOOK_OFFLINE=1 ./gradlew :book --offline`. `Gradle --offline` controls Gradle dependencies; `BOOK_OFFLINE=1` selects the cached TeX bundle for both Tectonic builds.

1.3 Understanding Binary Data and File I/O

A `ByteBuffer` is a byte view with a position and limit. Before reading, a buffer whose writing is complete usually needs `flip()`; whether to clear it after writing depends on whether it will be reused. The protocol and on-disk layout specify big-endian order, so both encoders and decoders must use the same byte order. A logical offset identifies a record; a position identifies a byte in a file. Even if their numeric values happen to match, they have no semantic relationship.

`FileChannel` can read and write at a specified file offset, and can request that written data be forced to storage. The course provides `ChannelIO.readFully/writeFully` for handling partial reads and writes: one `channel.read` is not guaranteed to fill a buffer, and one `write` is not guaranteed to write everything. Step 2 chooses `force(false)` after each batch append. This is a simplified request to persist file data, not a mathematical guarantee against power loss, device caches, media damage, or filesystem semantics. CRC32C detects accidental changes to record bytes; it does not repair errors or prove that data came from a trusted source.

1.4 How to Read Test Failures

First read the failed Step test name and assertion, then return to the contract in this chapter. The first exception in a test report is usually more useful than the last part of the stack trace: an unimplemented method identifies its step and method; course errors should preserve `ErrorCode`; an unexpected `IOException` may be wrapped as `STORAGE_ERROR` with its cause preserved. Do not map errors to an empty list or a successful default value just to make tests green. A single-step diagnostic command runs only that step and does not mean its cumulative dependencies are satisfied; the cumulative command reevaluates every earlier step. The course runner prints named tests grouped by step, per-step and overall counts, failure assertions with exact method navigation, and rerun commands. The report is always in English; legacy JUnit XML is saved under `build/student/reports/` or `build/reference/reports/`. These custom JavaExec tasks print plain text. Standard Gradle `:exercises:test` and `:reference:test` tasks support IDE

debugging and exact `--tests` selection; their XML/HTML reports use `build/<mode>/gradle-test-results/` and `build/<mode>/gradle-test-reports/`. The runner retains the `NO_COLOR` and `TERM=dumb` color gate.

Command	Meaning
<code>./gradlew :list</code>	Show all 31 steps, their editable methods, and prerequisites.
<code>./gradlew :stepTest -Pstep=12</code>	Diagnose Step 12 only; useful for locating the current problem when it fails.
<code>./gradlew :test -Pstep=12</code>	Run cumulative tests from <code>Step01Test</code> through <code>Step12Test</code> .
<code>./gradlew :referenceTest -Pstep=31</code>	Run the complete 31-step contract against the isolated reference implementation.
<code>./gradlew :referenceDemo</code>	Run the completed reference TCP/failover scenario.
<code>./gradlew :referenceConsistencyDemo</code>	Run the real TCP and disk consistency scenarios for Chapter 8.

Step numbers accept decimal values from 1 through 31, with optional leading zeroes (for example, `-Pstep=1` or `-Pstep=01`). Chapter boundaries are Steps 4, 8, 12, 16, 20, 24, 28, and 31: a chapter is complete only when cumulative tests through that boundary all pass. Passing `./gradlew :test -Pstep=8` means Steps 1–8 all pass; passing `./gradlew :stepTest -Pstep=8` means only the Step 8 diagnostic ran and does not authorize progression. Full-course tests must not put reference sources on the student classpath, and reference mode cannot substitute for exercise progress.

1.5 Start with System Invariants

Keep four boundaries in mind as you read the following chapters: first, offsets are continuous within the retained log range, LEO is the next write position, and fetch uses a half-open interval; second, the range visible to an ordinary consumer cannot pass HW, while a group’s committed offset is a separate state variable; third, locks are acquired in the order authority partition lock then local log lock, and network I/O never waits while holding that lock; fourth, errors must be meaningful, especially when an epoch or generation becomes stale—silent retries must not fabricate exactly-once behavior. Each chapter explains which new invariant it introduces and who verifies it.

These invariants take concrete form in later steps: the on-disk format is in [Step 1 \(Record encoding and decoding\)](#), log-tail boundaries in [Step 4 \(Crash recovery\)](#), consumer positions in [Step 14 \(Pull-based consumer\)](#), replication visibility in [Step 22 \(ISR and high watermark\)](#), and role fencing in [Step 25 \(Clean election and leader epochs\)](#).

Chapter 2

Records and Single-Segment Persistence

This chapter answers a basic but important question: after a message is written to a file, how can we know that it is complete, in order, and recoverable after a process restart? Conceptually, a Kafka partition is an append-only log. This course first encodes each record directly into a single-segment file, then adds segmentation, indexing, and retention. An offset is a logical position and cannot be replaced with a file byte position.

length	CRC32C	offset	timestamp	key/value len	key value
4 B	4 B	8 B	8 B	8 B	variable

Figure 2.1: A disk record. length does not include its own leading 4 bytes.

2.1 Step 1: Record Encoding and Decoding

Goals and prerequisites

Goal: Define stable on-disk boundaries so writers and readers assign the same meaning to the bytes.

Before you begin, be able to explain: Understand signed lengths, fixed-width integers, UTF-8 bytes, and CRC32C; prior knowledge of file scanning is not required.

Background and reasoning

The format is big-endian: `length:int32|crc32c:int32|offset:int64|timestamp:int64|keyLength:int32|valueLength:int32|key|value`. `length` is counted from the CRC field and has a legal range of 28 to 1,048,576; `keyLength=-1` means null, while 0 means an empty key. `value` must be present but may have length 0. CRC32C covers bytes from the beginning of `offset` through the end of `value`; neither the length prefix nor the CRC field itself participates in the check. Both `offset` and `timestamp` must be nonnegative. Validate the length before consuming bytes according to the declared fields so that a malicious or corrupt length can be distinguished from a genuinely short buffer. Decoding consumes exactly one record; even if another record follows, it must not consume the suffix.

Starting point and edit location.

Already provided: The `LogRecord`, `RecordData`, `CourseException`, and `ErrorCode` types, big-endian fields, and `CRC32C` utility types are provided. Array ownership is delegated to the data types; do not make duplicate copies for every field access.

Implement in: `exercises/src/main/java/io/simplekafka/storage/RecordCodec.java`: `encode(LogRecord)`, `decode(ByteBuffer)`.

Behavioral contract and invariants. Encoding must allocate exactly the required record space; a record’s offset and timestamp cannot be negative, and value cannot be null. Insufficient input for decoding is `INVALID_REQUEST`; a format error or CRC mismatch is `CORRUPT_RECORD`. On success, the buffer position is exactly after this record, leaving subsequent bytes untouched for the next decode. A null key must not be confused with an empty key.

Tiered hints (not the answer)

Level 1

Split the format table into a “fixed-width header” and a “variable-length tail,” and list the byte count of each field first.

Level 2

Make the CRC’s protected start and end boundaries explicit; the definition of length is not the same as the total number of allocated bytes.

Level 3

Write round-trip and single-byte corruption tests, then separately observe which protocol error should result from a short buffer, an illegal length, and a failed checksum.

Observable examples and test points. Encode and then decode a record with offset 7, a null key, timestamp 9, and value bytes equal to the UTF-8 encoding of “值”. The offset, key, timestamp, and bytes must be preserved. Flipping one bit in value makes decode report `CORRUPT_RECORD`; a buffer shorter than one complete record reports `INVALID_REQUEST`. These cases correspond to `Step01Test` and also verify that decoding does not consume a record immediately following this one.

Common pitfalls and differences from real Kafka

Counting the length prefix in length, including the CRC field in its own CRC, or interpreting a null key as an empty array all break interoperability. CRC detects corruption; it is not a security signature. Real Kafka usually organizes records into batches and has richer compression and batch formats. This course deliberately uses a single-record format so field boundaries are visible; it does not claim that this is Kafka’s disk format.

Commands and progression. Use `./gradlew :stepTest -Pstep=1` to diagnose this step; `./gradlew :test -Pstep=1` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can’t a record’s logical offset be inferred from its file byte position?
2. What can a successful CRC check prove, and what can it not prove?

2.2 Step 2: Log Append

Goals and prerequisites

Goal: Implement continuous offset allocation and observable persistence for a single-segment log.

Before you begin, be able to explain: Know the encoding from Step 1 and distinguish locking, full-write loops, and writes at file positions.

Background and reasoning

Appending turns data in memory into ordered log facts. The partition's local lock serializes appends to the same partition so competing batches do not receive interleaved or overlapping offsets. A single ordinary `FileChannel.write` may write only part of a buffer, so the provided `ChannelIO.writeFully` must be used. After writing a batch, call `force(false)` before publishing the append result; `AppendResult(firstOffset, nextOffset)` uses a half-open interval, for example `[0,3)` for three records. If an I/O error occurs during writing, the course does not guarantee atomic rollback of the whole batch; recovery after restart is responsible for identifying a physical tail fragment.

Starting point and edit location.

Already provided: The `SegmentLog` path, `FileChannel`, `lock`, `baseOffset`, `nextOffset`, and `close()` are given; record encoding and partial-write I/O can be reused.

Implement in: `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`: `append(List<RecordData>)`.

Behavioral contract and invariants. An empty batch reports `INVALID_REQUEST`. A nonempty batch receives continuous offsets starting at the current LEO while holding the lock; precheck all parameters and each record's size before writing the batch so known-invalid input cannot write a prefix and then fail. After all records have been written and `force` succeeds, return the first offset and the next offset. The established types and codec contract constrain each record's timestamp and data validity.

Tiered hints (not the answer)

Level 1

Express the result using the first and next offsets of a batch; do not mistake “record count” for the first offset of the next batch.

Level 2

Think about which errors can be detected before acquiring the lock and which state must be read while holding it.

Level 3

Have concurrent appends compete on the same log instance and observe batch-level ordering and file contents; do not assert only the final LEO.

Observable examples and test points. Append three records and then two records to an empty segment; the expected results are `[0,3)` and `[3,5)`, respectively, and decoding the file yields offsets 0, 1, 2, 3, and 4. The result intervals for two concurrent batches of two records must not overlap, and the total record count must be four. These are the actual-file and concurrency checks in `Step02Test`.

Common pitfalls and differences from real Kafka

Writing first and discovering that a record is too large later leaves an invalid partial batch. Updating `nextOffset` without forcing or checking the write is also wrong. Real Kafka uses batching,

background I/O, and more complex persistence policies; this course explicitly forces after every append so the boundary is observable, not to model Kafka’s performance policy of synchronously writing every message to disk.

Commands and progression. Use `./gradlew :stepTest -Pstep=2` to diagnose this step; `./gradlew :test -Pstep=2` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should the entire batch be validated before any disk write?
2. What does a successful force mean, and what does it not mean?

2.3 Step 3: Offset-Based Reads

Goals and prerequisites

Goal: Read complete records from a specified offset while honoring both record-count and byte budgets.

Before you begin, be able to explain: Distinguish offsets, in-segment byte positions, record integrity, and cursor state.

Background and reasoning

Consumers request logical offsets, while file reads require physical byte positions. The first version of the public read starts at the beginning of the segment; the index in the next chapter will provide the log with a physical-position hint near a logical offset. A hint can shorten the search but cannot change offset semantics. Each record is an indivisible return unit: if the next complete record does not fit the budget, do not return part of it. The budget includes the length prefix. Explicit-position `FileChannel` reads avoid sharing a mutable channel cursor between reads and appends.

Starting point and edit location.

Already provided: Segment lifecycle management and record decoding are complete; the public `read(offset,maxRecords,maxBytes)` passes a default hint to the internal method.

Implement in: `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java`: `readFrom(longoffset,longbytePositionHint,intmaxRecords,intmaxBytes)`.

Behavioral contract and invariants. `offset` may range from `baseOffset` through LEO (reading at LEO returns empty); an offset outside the segment’s valid range reports `OFFSET_OUT_OF_RANGE`. `maxRecords` and `maxBytes` must be positive. Return records with continuous offsets that do not exceed either budget; if the first record alone exceeds the byte budget, return an empty result. A valid hint must be at a record boundary and cannot pass the target offset, or report `INVALID_REQUEST`. Reads must not move the append position or change LEO.

Tiered hints (not the answer)

Level 1

Treat “find a candidate byte position” and “which offset should the output start at?” as two separate questions.

Level 2

For the byte budget, calculate the size of each complete record, including the length prefix; do not approximate it using payload bytes.

Level 3

Design one hint that points to a boundary before the target and another that points past the target, then verify the conditions for accepting and rejecting them.

Observable examples and test points. After appending offsets 0–4, `read(2,2,4096)` returns offsets 2 and 3. Then call `readFrom(2,hint,2,4096)` with hint set to the actual starting byte position of the offset 1 record in the file; it should still return offsets 2 and 3. The hint is a physical position, not the numeric logical offset. `read(5,2,4096)` returns empty. Reading from offset 1 with a 66 byte budget returns exactly 1 and 2; a smaller budget returns only 1. If the first complete record exceeds `maxBytes`, return empty rather than partial bytes. These cases correspond to `Step03Test`.

Common pitfalls and differences from real Kafka

Offset 2 is not file byte 2; confusing them will randomly drop records once an index is introduced. Ignoring the prefix in the budget exceeds the caller’s memory budget. A real Kafka fetch usually negotiates batch and network-byte limits; this implementation returns one record atomically, and the next chapter handles the total budget across segments.

Commands and progression. Use `./gradlew :stepTest -Pstep=3` to diagnose this step; `./gradlew :test -Pstep=3` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can a sparse index provide only a search starting point, not directly determine the read result?
2. What problem would returning a partial record cause when the first record is larger than `maxBytes`?

2.4 Step 4: Crash Recovery**Goals and prerequisites**

Goal: Recover a continuous LEO from a log that may have closed partway through a write, and repair only cases that can be proven to be an incomplete tail.

Before you begin, be able to explain: Understand record boundaries, physical truncation, and error categories.

Background and reasoning

A crash can occur while writing the length prefix, fixed header, or payload. If EOF falls inside the last record’s header or body, data before the last complete record can still form a valid log, so the file can be truncated to the preceding record boundary. However, a complete record with a bad CRC, an illegal length, or a discontinuous offset is not something that can be “safely discarded as a tail.” Silently truncating complete but damaged evidence would report a media failure as successful recovery. Recovery starts with the expected offset set to `baseOffset`, scans to the last valid boundary, and returns the next writable position.

Starting point and edit location.

Already provided: `SegmentLog.open(file,baseOffset)` opens an existing file and invokes recovery; the codec from Step 1 and file layout from Steps 2 and 3 are established.

Implement in: `exercises/src/main/java/io/simplekafka/storage/SegmentLog.java:recover()`.

Behavioral contract and invariants. On success, return the recovered LEO so the next append continues from it; only an incomplete record at the end of the file may be truncated. A complete record with a bad CRC, illegal length, or discontinuous offset must fail with `CORRUPT_RECORD`; do not skip the bad record and continue scanning. Underlying I/O exceptions use `STORAGE_ERROR` and preserve the cause.

Tiered hints (not the answer)**Level 1**

First define the “shared state” after each successful scan: the current valid byte boundary and the next expected offset.

Level 2

Classify EOF with too few bytes separately from a fully read record whose contents are incorrect.

Level 3

Construct two reopen tests: one with a partial tail record and one with a complete record whose CRC is corrupt. Confirm that only the former changes the file length.

Observable examples and test points. The file contains complete offsets 0 and 1, followed by a partial body for offset 2. After reopening, `LEO=2` and the file is truncated exactly to the end of the second record; appending one record gives offset 2. If offset 1 is complete but its CRC is wrong, reopen must fail rather than treating it as a tail fragment. These cases correspond to `Step04Test`.

Common pitfalls and differences from real Kafka

“Truncate whenever scanning reports an error” swallows corruption and format errors in complete records; “any EOF makes the file invalid” cannot handle a normal crash tail. `force(false)` expresses this course’s write intent; it is not proof of power-loss safety. Real Kafka also has segment recovery, index repair, log validation, and more complex cleanup policies; this step recovers only the boundary of one segment.

Commands and progression. Use `./gradlew :stepTest -Pstep=4` to diagnose this step; `./gradlew :test -Pstep=4` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Which byte-level evidence is enough to identify an incomplete tail, and which evidence must be reported as corruption?
2. Why should recovery return LEO rather than the offset of the last record?

Chapter 3

Segments, Indexing, Retention, and Truncation

As a single-file log grows, startup scans become slower, and old data cannot be deleted on its own. Kafka-style logs use multiple segments for a partition: old segments are closed, and the active segment receives appends; a sparse index maps logical offsets to byte boundaries within a segment. The index is a rebuildable acceleration structure, not the source of record truth. Log retention and truncation are different operations: retention deletes an old closed prefix, while truncation moves the tail back to an offset to repair a divergent suffix.

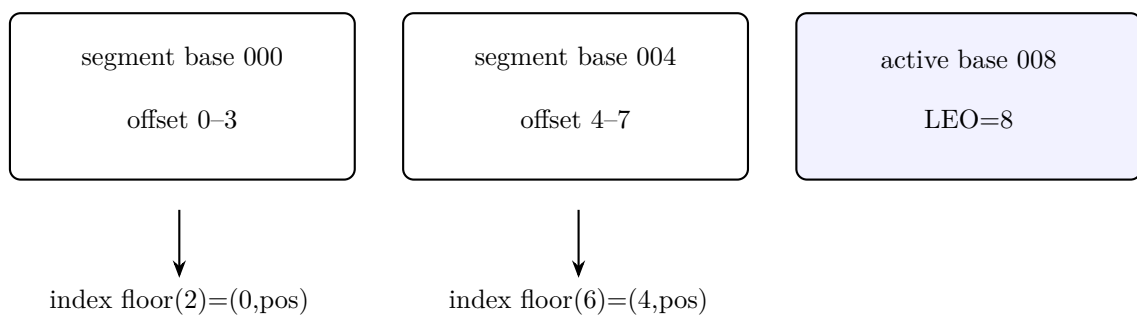


Figure 3.1: Logical ranges in a segmented log and the sparse index within each segment. The base offset in a segment filename is its logical starting point.

3.1 Step 5: Sparse Index

Goals and prerequisites

Goal: Build a recoverable sparse mapping from offsets to byte positions.

Before you begin, be able to explain: Understand floor lookup, record boundaries, and recovery by scanning the log.

Background and reasoning

Each index entry consists of two big-endian int64 values: `offset, position`; each entry is 16 bytes. Write one point every `intervalRecords` records, and always index the first record. When looking up a target offset, `floor` returns the greatest offset not larger than the target, allowing the read to start at a nearby valid boundary and scan forward. If no suitable entry exists, `(baseOffset, 0)` is a safe starting point. An index may be missing, truncated, or contain invalid

entries; the main log remains the source of truth, so recovery should scan the log to rebuild the index.

Starting point and edit location.

Already provided: The index file path, segment baseOffset, interval, and lifecycle are provided by `SparseIndex`; `SegmentLog` can rescan records.

Implement in: `exercises/src/main/java/io/simplekafka/storage/SparseIndex.java`:
`add(long, long), floor(long), rebuild(Path, long).`

Behavioral contract and invariants. Add entries in strictly increasing offset order with nonnegative positions; `floor` returns the greatest offset $\leq$ target, or the segment start at byte position zero if no entry is found. If the index is missing, its file length is not a multiple of 16, or an entry is invalid, rebuild it from the log; corruption in the log itself must still fail. Rebuilding must not make an invalid index look like trusted log data.

Tiered hints (not the answer)

Level 1

First write down the mathematical definition of `floor`: what offsets are in the candidate set, and what is the convention for an empty set?

Level 2

Separate index-read failures into “the index can be discarded” and “the primary log cannot be recovered.”

Level 3

Create records across multiple segments with an interval index, delete the index file, and observe recovery through a real read rather than checking only an in-memory map.

Observable examples and test points. With `interval=4`, create entries for offsets 0, 4, and 8; `floor(6)` returns the physical position associated with offset 4. Delete `.index`, reopen, and read offset 6; the correct record is still returned. An index file with an invalid length can also be rebuilt, while a corrupt log still reports an error. These cases correspond to `Step05Test`.

Common pitfalls and differences from real Kafka

If an index entry points inside a record, the reader will interpret arbitrary payload bytes as a header; using ceiling instead of floor skips the boundary before the target. If the index becomes the sole data source, a missing index loses the log. Kafka’s index organization and precision/byte-offset model are more complex; this course indexes once every `intervalRecords` records, not at a fixed byte interval.

Commands and progression. Use `./gradlew :stepTest -Pstep=5` to diagnose this step; `./gradlew :test -Pstep=5` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must records still be scanned after finding a floor entry?
2. Why can’t a corrupt index and a corrupt log use the same recovery strategy?

3.2 Step 6: Segmented Log

Goals and prerequisites

Goal: Combine single-segment logs into a rolling partition log and make reads work continuously across segments.

Before you begin, be able to explain: Understand segment boundaries, capacity estimation, indexes, and the global LEO.

Background and reasoning

PartitionLog holds multiple SegmentLog instances sorted by baseOffset. If the active segment cannot fit the next valid record, close it and create a segment whose base is the current LEO; even if one record is larger than the configured segmentBytes, it is valid as long as the record itself is legal and should occupy a new segment on its own rather than remain impossible to append. A batch may cross segment boundaries, while offsets remain continuous across the entire partition. A read first locates the target segment, then uses that segment's index floor as a hint, scans forward, and enters the next segment if needed, treating maxRecords and maxBytes as total budgets across segments. Precheck the validity of the entire batch before creating new files so known-invalid input does not change the directory structure. After the whole-batch precheck, increment the version once before the first disk write or segment roll; even a write attempt that later encounters an I/O error has already invalidated an old proof.

Starting point and edit location.

Already provided: Segment creation/open/recovery, sparse indexing, and resource closing are provided for each segment. The PartitionLog skeleton provides mutationVersion() and private markMutation(); call it at the correct point when implementing append.

Implement in: `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`: `append(List<RecordData>)`, `read(long,int,int)`.

Behavioral contract and invariants. `append` returns the `[first,next)` interval for the entire batch, with no gaps in offsets across segments. Roll a segment before appending only when capacity is insufficient; an invalid batch must not roll first. Reads traverse segments in offset order and accumulate the total record and byte limits; a direct append to a segment does not itself roll it. Reopening an existing directory recovers LEO without allocating old offsets again. Directory discovery handles only the designated `.log/.index` files and does not delete unfamiliar files. Each valid append batch increments mutationVersion once on the open log object; this is only an in-memory generation, not a disk field.

Tiered hints (not the answer)

Level 1

Base the capacity check on the complete encoded record size, not just the value length.

Level 2

Decide how each record in a batch relates to segment boundaries while keeping global offsets continuous.

Level 3

Separate tests for the initial append, append after a roll, and append after close/reopen, then verify every offset one by one.

Observable examples and test points. Configure 256-byte segments and append ten records, close and reopen, then append one more. Reads should still cover offsets 0 through 10 without duplicate offsets. Tests also cover a batch crossing a segment boundary, a single record occupying its own segment, and directory/LEO remaining unchanged before and after an invalid oversized record. These cases correspond to `Step06Test`.

Common pitfalls and differences from real Kafka

Using only raw payload size for capacity omits the fixed header, length, and CRC; checking offsets separately for each batch can create conflicts across segments. Real Kafka segments more often roll by byte size and combine index files with time-based policies; this course also rolls by capacity, but the index interval is a record count. A single-segment append intentionally does not roll segments.

Commands and progression. Use `./gradlew :stepTest -Pstep=6` to diagnose this step; `./gradlew :test -Pstep=6` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must the entire batch be validated before creating any new segment?
2. How can a cross-segment fetch avoid resetting its byte budget when moving between segments?

3.3 Step 7: Log Retention

Goals and prerequisites

Goal: Safely delete expired closed segments while retaining the active segment and current LEO.

Before you begin, be able to explain: Distinguish the log start, consumer progress, and deletion of individual records.

Background and reasoning

The partition log has only one retained range: `[logStartOffset, LEO)`. This course performs retention through an explicit call; it does not run a background timer. `deleteBefore(offset)` deletes only closed segments whose last record offset is less than the threshold; it cannot split a segment by record and cannot remove the active segment. The new start must lie on an actual segment boundary. A consumer requesting data that has already been deleted receives `OFFSET_OUT_OF_RANGE`; its position is not silently changed to the new start. Consumer-group offset commits are independent of physical log deletion, and advancing a consumer position does not trigger retention automatically. Increment this log object's `mutationVersion` once only when at least one segment is actually deleted; a call with an invalid threshold or no deletable segment leaves the version unchanged.

Starting point and edit location.

Already provided: The multi-segment directory scan, log start, and segment-closing path are held by the `PartitionLog` skeleton; `mutationVersion()` and private `markMutation()` are provided.

Implement in: `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`: `deleteBefore(long)`.

Behavioral contract and invariants. Delete only qualifying closed segments and the `.index` with the same base; do not delete the active segment or unfamiliar files. Return the actual new `logStartOffset`. LEO does not move backward; after reopening, the new start remains correct. If an existing read position is below the new start, the read must report an explicit range error and must not seek automatically. Mark once before deletion begins; a no-op does not increment `mutationVersion`.

Tiered hints (not the answer)**Level 1**

Use segment bases and the next segment's base to determine which logical offsets belong to the deletable prefix.

Level 2

Consider an empty active segment and a nonempty active segment separately; do not treat "the last segment" as deletable.

Level 3

After deletion, close and reopen, then read from both before and after the boundary to verify that disk files and the in-memory start agree.

Observable examples and test points. Create three segments, retain the last active segment, and delete old closed segments; `logStartOffset` moves to the retained boundary. Reopening returns the same start and leaves LEO unchanged. A read below the new start reports `OFFSET_OUT_OF_RANGE`. These cases correspond to `Step07Test`.

Common pitfalls and differences from real Kafka

Deleting only `.log` and not its paired index creates a dangling index; deleting the active segment based on the threshold loses the append starting point. Real Kafka retention is often triggered in the background by time/size configuration and may relate to consumer progress independently; this course provides only an explicit synchronous call and does not reset consumer positions.

Commands and progression. Use `./gradlew :stepTest -Pstep=7` to diagnose this step; `./gradlew :test -Pstep=7` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why does a deletion threshold usually not coincide exactly with a segment boundary for a record?
2. If a committed position has passed an old segment, does that mean the system can delete it automatically? Why or why not?

3.4 Step 8: Tail Truncation**Goals and prerequisites**

Goal: Truncate the log tail back to a next-record offset, remove divergent tail data, and restore an appendable state.

Before you begin, be able to explain: Understand half-open intervals, record boundaries, and segment bases.

Background and reasoning

The truncation target `nextOffset` means retain all records with `offset < nextOffset`; the valid range is `[logStartOffset, LEO]`. If the target is exactly at an existing segment boundary, an active segment must begin at that base so appends can continue. If the target is within a segment, truncate at the physical boundary of that record, delete later segments, and rebuild the retained segment's index. Truncating to LEO is a no-op; truncating to `logStartOffset` leaves an empty

segment, and the next append uses that offset. Truncation changes only the local log tail; it does not automatically change replication state, HW, or consumer positions. Before the first write or deletion, increment this object’s `mutationVersion` once for a valid non-no-op truncation; invalid calls and no-ops do not increment it.

Starting point and edit location.

Already provided: Segment closing, file truncation, index rebuilding, and the global segment table are established in earlier steps. The `PartitionLog` skeleton provides `mutationVersion()` and `private markMutation()`; a valid truncation must call it before the first file change.

Implement in: `exercises/src/main/java/io/simplekafka/storage/PartitionLog.java`: `truncateTo(long)`.

Behavioral contract and invariants. Only `nextOffset` values in `[logStartOffset, LEO]` are allowed; an out-of-range value is rejected and the log remains unchanged. Retain offsets `<nextOffset` and remove all later data and its indexes; update LEO to `nextOffset`. After success, an append can happen immediately and the new record’s offset is exactly `nextOffset`. Unfamiliar files cannot be deleted. Invalid parameters and truncating to the current LEO do not increment `mutationVersion`; mark any valid content change before the first I/O.

Tiered hints (not the answer)

Level 1

Think of the target as “the offset to write next,” not the offset of the last record to retain.

Level 2

List the disk state for truncation inside a segment, at a segment boundary, at the current LEO, and at `logStartOffset`.

Level 3

Reopen the truncated directory and append one record; checking only the in-memory `nextOffset` is not enough to prove that indexes and files agree.

Observable examples and test points. After writing offsets 0–8, run `truncateTo(5)`; offsets 0–4 remain readable, offsets 5–8 are not returned, and reading the new LEO 5 yields an empty result. The next append receives offset 5. `truncateTo(LEO)` leaves contents unchanged; truncating to the start retains an empty active segment and appends from that start. These cases correspond to `Step08Test`.

Common pitfalls and differences from real Kafka

Misreading `nextOffset` as the last retained record creates an off-by-one error; truncating the file without rebuilding the index can make later reads jump to deleted divergent records. Real Kafka truncation is often coordinated with leader epochs, HW, and replica protocols; this step provides only a log primitive and cannot by itself establish a safe election.

Commands and progression. Use `./gradlew :stepTest -Pstep=8` to diagnose this step; `./gradlew :test -Pstep=8` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. When truncating to 5, why should the next record be written at offset 5 rather than 4?
2. Why must the index file also be handled after cleaning the file tail?

Chapter 4

Partitions and a Broker on a Real Network

The log in the previous chapter had only local meaning; this chapter creates multiple partitions for a topic, then separates clients from the broker with real loopback TCP requests. Partitions define ordered ranges of records, and topic metadata tells clients the leader of each partition. TCP is a byte stream and does not preserve the boundaries of Java writes, so the protocol must include frame lengths. This course implements only its teaching binary messages and is not compatible with the Kafka wire protocol.

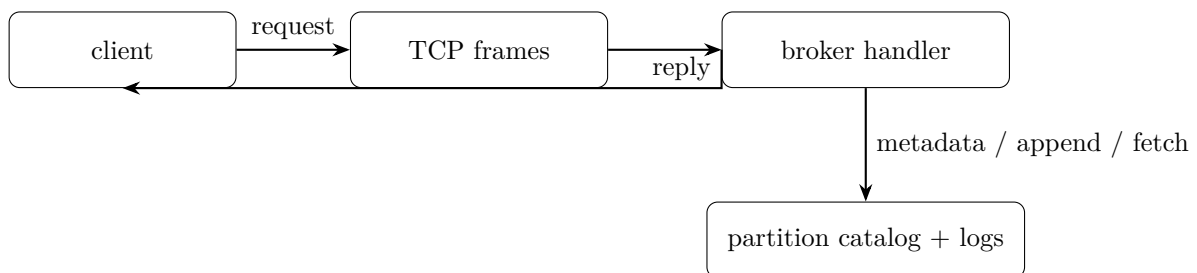


Figure 4.1: Request/response path over one connection. Provided code handles message-body encoding and decoding; students implement frames and business dispatch.

4.1 Step 9: Partition Routing

Goals and prerequisites

Goal: Route the same key to the same partition consistently while providing observable rotation for null keys.

Before you begin, be able to explain: Understand key byte identity, CRC32C, and modulo.

Background and reasoning

A producer must choose a partition before it can establish ordering within that partition. For a non-null key, this course computes CRC32C and takes the CRC as an unsigned value modulo the partition count; the same byte string therefore consistently selects the same partition. A null key has no key to hash, so one Partitioner instance rotates, starting at 0. An empty byte array is a present key of length zero, not the same as null. The partition count must be positive. This simple rule makes results deterministic and easy to observe.

Starting point and edit location.

Already provided: The partitioner's round-robin state and no-argument constructor are provided; topic metadata supplies the partition count.

Implement in: `exercises/src/main/java/io/simplekafka/client/Partitioner.java`: `choose(byte[]key,intpartitions)`.

Behavioral contract and invariants. `partitions<=0` is an invalid request. Routing a non-null key uses the unsigned interpretation of its CRC32C; before each null-key selection, normalize the per-instance cursor modulo the current partition count, so a topic-size change still returns a valid partition. Different instances each start from zero.

Tiered hints (not the answer)**Level 1**

First distinguish three inputs: null, an empty array, and a nonempty array.

Level 2

Check the conversion semantics between Java's signed int and the problem's unsigned CRC so that a negative remainder cannot become a negative partition.

Level 3

Mix calls with a key, a null key, and different partition counts on the same instance; examine whether the round-robin cursor advances only for the specified inputs.

Observable examples and test points. With 3 partitions, the same key selects the same partition 100 times in a row; an empty key is also routed deterministically. A fresh Partitioner returns 0, 1, 2, 0, 1, 2 for six consecutive null-key calls. With counts 3,3,1,3,3,2,2, null-key choices are 0,1,0,0,1,0,1. These cases correspond to `Step09Test`.

Common pitfalls and differences from real Kafka

Using a key's `hashCode` instead of CRC32C, or taking a remainder directly from a value that may be negative, violates the routing contract. Real Kafka's default partitioner uses a different key-hash algorithm and a sticky-batching strategy for null keys; this course uses instance-level round-robin for null keys, which must not be described as Kafka's default behavior.

Commands and progression. Use `./gradlew :stepTest -Pstep=9` to diagnose this step; `./gradlew :test -Pstep=9` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should the hash be computed over key bytes rather than a `RecordData` object's Java `hashCode`?
2. How do a null key and a zero-length key array differ in the protocol and in routing?

4.2 Step 10: Topic Metadata

Goals and prerequisites

Goal: Register a topic's fixed partition set in a safe directory structure and let clients discover leaders in a deterministic order.

Before you begin, be able to explain: Understand partition metadata and the purpose of idempotent creation.

Background and reasoning

Metadata maps logical names to physical partitions. Each partition carries at least a TopicPartition, leader id, epoch, replica/ISR lists, and Endpoint; in single-broker mode, epoch is 0 and replicas and ISR contain only the current broker. The number of partitions for a topic is fixed when it is created in this course catalog. The directory structure is root/topic/partition. Topic names are restricted to `[A-Za-z0-9._-]+`, with `.` and `..` rejected to prevent path traversal. On directory restart, explicitly create the same topic/count again to open existing logs; do not infer the total partition count from files that happen to exist.

Starting point and edit location.

Already provided: Endpoint, partition metadata value types, and log registration paths are provided; the Catalog constructor fixes the broker id and endpoint.

Implement in: `exercises/src/main/java/io/simplekafka/broker/PartitionCatalog.java`: `createTopic(String,int), metadata(String)`.

Behavioral contract and invariants. A positive partition count creates a fixed set of partitions; creating the same topic with the same count is idempotent, while a different count reports `INVALID_REQUEST`. An unknown topic reports `UNKNOWN_TOPIC_OR_PARTITION`. metadata returns entries in increasing partition-id order and does not expose paths outside root. Single-broker metadata has `epoch=0` and `replicas/isr=[brokerId]`.

Tiered hints (not the answer)

Level 1

Creating a topic is more than `mkdir`: consider the partition count and constraints that can be verified after a restart.

Level 2

Give same-name/same-count calls and same-name/different-count calls different semantics.

Level 3

Test special path characters, empty names, and reopening directories to confirm that name validation occurs before file creation.

Observable examples and test points. After creating `orders` with 2 partitions, metadata contains `orders-0` and `orders-1` in a fixed order; creating the same definition again is harmless, while changing it to 3 reports an error. Querying an unknown topic returns the topic/partition error code. These cases correspond to `Step10Test`.

Common pitfalls and differences from real Kafka

Concatenating a user topic directly into a path exposes the disk boundary to path traversal; enumerating directories to recover the partition count can treat manual leftovers as metadata. Real Kafka topic metadata is managed by a cluster control plane, and partitions can be expanded through controlled operations; this course catalog supports only an explicitly fixed count and does not handle dynamic expansion.

Commands and progression. Use `./gradlew :stepTest -Pstep=10` to diagnose this step; `./gradlew :test -Pstep=10` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can't "same name but different partition count" be treated as idempotent success?

2. What evidence can a disk directory provide, and what metadata can it not replace?

4.3 Step 11: TCP Frame Protocol

Goals and prerequisites

Goal: Split a TCP byte stream into bounded request/response frames that can be matched.

Before you begin, be able to explain: Understand `readFully`, EOF position, length prefixes, and correlation ids.

Starting point and edit location.

Already provided: Messages request/success-response bodies, API id constants, the Frame value type, and ErrorCode wire ids are provided; this step does not rewrite the body codec.

Implement in: `exercises/src/main/java/io/simplekafka/protocol/FrameCodec.java`:
`write(OutputStream, Frame)`, `read(InputStream)`.

Behavioral contract and invariants. EOF before the first byte returns null; EOF partway through a frame is `INVALID_REQUEST`. Reject negative or oversized lengths before allocating. The server validates and rejects unknown version/API values and nonzero request errors. Read consecutive frames from a stream one at a time without losing boundaries; never send a partial frame. Stream I/O throws `IOException` as specified by the interface, while protocol-format errors use course error codes. If a response body cannot fit its frame, return a bounded `INVALID_REQUEST` error reply instead of silently closing the connection.

Tiered hints (not the answer)

Level 1

The minimum frame length is the sum of the widths of the fixed header fields, not something to infer from an example payload.

Level 2

Handle “clean end of stream” and “end of stream inside a frame” separately.

Level 3

Feed two consecutive frames through an `InputStream` that returns only one byte per read, then check immediate rejection using input that contains only an oversized length prefix.

Observable examples and test points. Split a two-frame stream into single-byte chunks; two consecutive reads return the original correlation ids and payloads, and only the next read returns null. Input with only a 4-byte oversized length and no body fails immediately instead of waiting or allocating several megabytes. These cases correspond to `Step11Test`.

Common pitfalls and differences from real Kafka

A single read that is expected to fill a header is vulnerable to TCP segmentation; allocating before checking length lets a malicious request amplify memory use. Real Kafka has its own request headers, API-version negotiation, and message formats. This course’s version=1 and API list define a teaching protocol and cannot be used with official clients.

Commands and progression. Use `./gradlew :stepTest -Pstep=11` to diagnose this step; `./gradlew :test -Pstep=11` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must a `correlationId` be returned with its corresponding request?
2. What is the value of distinguishing “normal EOF” from “half-frame EOF” in a stream?

4.4 Step 12: Single-Broker Requests**Goals and prerequisites**

Goal: Connect metadata, produce, and fetch business requests to the local catalog and verify them end to end over real TCP.

Before you begin, be able to explain: Know framing, body codecs, and single- and segmented-log steps.

Background and reasoning

The broker is the request-semantics boundary: validate the topic, partition, epoch, and parameters first; then call the log and finally wrap the successful result or a standard error in a Reply. In single-broker mode, log end offset is also the high watermark, so consumers can read immediately after an append. `produce` returns an append interval; `fetch` reads records using an offset and both budgets, and returns `logStart`, `LEO`, `HW`, and epoch. Unknown APIs are rejected at the transport layer; a business error affects only the current request, and the broker should continue accepting subsequent TCP requests. A real TCP call tests client framing and the server handler together, rather than calling methods directly.

Starting point and edit location.

Already provided: `BrokerHarness.single` manages log registration, TCP transport, `MessageCodec`, request-body parsing, and service lifecycle; `SingleBrokerMain` uses loopback and an OS-allocated port.

Implement in: `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: `handle(Request)`.

Behavioral contract and invariants. First implement `METADATA/PRODUCE/FETCH`.

An unknown partition has error `UNKNOWN_TOPIC_OR_PARTITION`; single-broker mode accepts only epoch 0, and offset-range errors retain their corresponding error codes. A successful produce returns `AppendResult`; a successful fetch does not pass `LEO=HW`. An error Reply contains only an error code and a short message, never a stack trace on the wire. The course `RpcClient` opens a new TCP connection for each request; one business error must not stop the broker from accepting later requests.

Tiered hints (not the answer)**Level 1**

Dispatch by API type, and complete each request’s identity and range checks before accessing the log.

Level 2

Check whether an error occurs in the transport, handler, or log layer; responsibilities at different layers must not be swallowed by one another.

Level 3

Use a real loopback connection to produce and fetch first, then send a failing request and continue with a valid request.

Observable examples and test points. Produce three records over TCP and receive their first/next interval; fetch the same partition and receive three offsets/keys/values with logStart=0 and LEO=HW=3. An unknown partition returns the correct error code, after which a valid fetch over a new TCP request still succeeds. These cases correspond to **Step12 Test**; you can also observe them manually with **SingleBrokerMain** and **ClientMain** on the command line.

Common pitfalls and differences from real Kafka

Calling only the handler's unit method cannot expose frame-boundary or correlation-id errors; converting an exception into an empty successful response hides errors and breaks client semantics. A real Kafka broker also has authentication, quotas, request-version compatibility, and a persistent control plane; this course's broker binds to loopback, and its API is incompatible with official clients.

Commands and progression. Use `./gradlew :stepTest -Pstep=12` to diagnose this step; `./gradlew :test -Pstep=12` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should a broker error reply not expose a Java stack trace?
2. Why does single-broker HW=LEO not mean the same rule can be used in replica mode?

Chapter 5

Producing, Consuming, Offsets, and Redelivery

This chapter turns the teaching program that calls the broker directly into understandable client behavior. The producer buffers messages by partition, the consumer pulls under explicit assignment, and `OffsetStore` writes processed progress to a local log. The key distinction is among three positions: the broker’s LEO, HW in replication mode, and a group’s committed offset. A poll advances the client’s in-memory position; that is not a commit. A commit also does not mean that broker log data has been deleted. The client takes ownership of the `RpcClient` passed directly to it and closes it on close; the `MetadataRouter` passed in is a borrowed shared dependency, and its creator should close it only after all clients using it have been closed.

5.1 Step 13: Batched Producer

Goals and prerequisites

Goal: Build per-partition ordered batch buffering, threshold-based sending, and explicit flush.

Before you begin, be able to explain: Understand metadata routing, batch boundaries, returned receipts, and uncertainty about request outcomes.

Background and reasoning

A batch for each partition must preserve call order, but different partitions can accumulate independently. `send` obtains the partition from metadata and synchronously sends that partition’s batch when it reaches `batchRecords`; `flush` sends all non-empty batches in stable order and aggregates `ProduceReceipt` values. The receipt returned by an automatic `send` must not be discarded: retain it and return it to the caller at the next `flush`. The default ACK is LEADER, and there is no background timer. A timeout or error during the produce RPC/reply path leaves the result unknown: the broker may already have appended the records. Any failure after that path begins enters an “outcome unknown” state; subsequent operations fail until close, rather than guessing and replaying.

Starting point and edit location.

Already provided: `RpcClient` manages sockets, timeouts, correlation ids, and requests/responses; metadata, `Partitioner`, and the batch record structure are provided.

Implement in: `exercises/src/main/java/io/simplekafka/client/SimpleProducer.java`
a: `send(String,byte[],byte[],long)`, `flush()`.

Behavioral contract and invariants. Preserve send order within the same partition; batches for different partitions may be independent. Send a partition's batch synchronously when it is full. A receipt stores the topic-partition and [firstOffset,nextOffset]; explicit flush returns receipts for both automatic sends and sends performed by that flush. Once the produce RPC/reply path begins, any failure leaves the append outcome unknown: preserve the first error and never replay the batch. Later send, flush, or `setAcks` calls report `REQUEST_TIMEOUT` until the producer is closed. Failures during metadata lookup or request preparation before that path do not poison the producer.

Tiered hints (not the answer)

Level 1

First distinguish “send the request,” “receive confirmation,” and “return the receipt to the caller” as three different points in time.

Level 2

Use `TopicPartition` as the buffer key, and check flush order and behavior for empty batches.

Level 3

Make the broker append and then return an error, or delay its response until timeout; verify that the client does not create a second copy.

Observable examples and test points. Configure 3 partitions and `batchRecords=2`. Consecutive records with the same key should route to the same partition and preserve order; null keys should be distributed by per-instance round robin. Reaching two records sends once, and flush returns results that include the automatic-send receipt. After an append-then-error reply, preserve that first error; later send, flush, and `setAcks` report `REQUEST_TIMEOUT` without replay. See `Step13Test` and `Step27Test`.

Common pitfalls and differences from real Kafka

If the receipt from an automatic flush is discarded, the caller cannot know which offset range already succeeded; if an RPC/reply failure is treated as proof of no append and retried, duplicates may result. This chapter's `SimpleProducer` deliberately sends synchronously and never retries automatically. Chapter 8 adds a separate `IdempotentProducer` whose caller can explicitly retry a batch with a stable producer identity; that does not change the ordinary producer's unknown-outcome contract.

Commands and progression. Use `./gradlew :stepTest -Pstep=13` to diagnose this step; `./gradlew :test -Pstep=13` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is a timeout insufficient to prove that the broker did not append the records?
2. If an error response for the same batch is retried, how could the caller identify a duplicate? Why does this course not implement that mechanism?

5.2 Step 14: Pull-Based Consumer

Goals and prerequisites

Goal: Build a single-threaded, single-round pull consumer that explicitly manages assigned partitions and local positions.

Before you begin, be able to explain: Understand seek, empty fetches, and the shared total budget.

Background and reasoning

A consumer's position is the offset of the next record to process or fetch. Manual assign deduplicates and sorts partitions, then replaces the old assignment with a new map: a retained partition keeps its previous position, a revoked partition is removed, and a new partition starts at position=0; it does not read OffsetStore. seek can replay or jump, but does not commit. poll is one bounded request, not a continuous long poll; all partitions share the maxRecords/maxBytes total budget. After records are returned successfully, only the corresponding partitions' positions advance to the last returned offset+1; a failed fetch for a partition must not advance that partition's position. An empty result means that no data is currently readable at that position; it does not mean the consumer should skip ahead. The consumer is used single-threadedly to avoid races on position from concurrent polls.

Starting point and edit location.

Already provided: RpcClient already provides FETCH; SimpleConsumer holds the group name and partition positions, and provides read-only position(tp). Step 14 does not call OffsetStore.

Implement in: `exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java`
 a: `assign(Collection)`, `seek(TopicPartition, long)`, `poll(int, int)`.

Behavioral contract and invariants. assign deduplicates and stably sorts; positions of retained partitions do not change, new partitions start at 0, revoked entries are removed, and committed offsets are not read. seek requires its target partition to be assigned, or reports NOT_ASSIGNED. After a successful poll, only partitions represented in returned records advance; an empty fetch changes nothing, and a failed partition does not advance. Reads do not commit automatically. Total records/bytes must not exceed request limits; the broker caps oversized positive maxBytes to a conservative single-frame budget, and subsequent fetches continue from the first unreturned offset. The returned mapping is organized in predictable partition order.

Tiered hints (not the answer)

Level 1

Model “assigned,” “has a position,” and “has a committed offset” separately; do not mix the store into the early manual mode.

Level 2

Maintain one remaining poll budget while reading the partitions instead of resetting it for each partition.

Level 3

Make a request fail for one of two partitions, then check how the positions of the other, successful partition and the failed partition change.

Observable examples and test points. After assigning two partitions, both initial positions are 0; poll reads offsets 0 and 1, then position=2; seek to 0 and poll again to reread 0. An empty fetch leaves position unchanged, and seek on an unassigned partition reports NOT_ASSIGNED. See Step14Test.

Common pitfalls and differences from real Kafka

Records returned by poll have not necessarily been durably committed; seek also changes only the local position. Using the full maxBytes for every partition can exceed the total budget. A real

Kafka consumer also provides long polling, thread restrictions, and a complex assignment/position lifecycle; this course uses one round at a time and explicitly assumes single-threaded use.

Commands and progression. Use `./gradlew :stepTest -Pstep=14` to diagnose this step; `./gradlew :test -Pstep=14` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why does advancing position after a successful read still not mean that a restart will begin at the new position?
2. If a request for one partition fails, why must the positions of all partitions not be advanced together?

5.3 Step 15: Consumer Offset Persistence

Goals and prerequisites

Goal: Persist the offset of the next record to process and restore it in a new consumer.

Before you begin, be able to explain: Distinguish physical log offsets, the group/tp key, and the message protocol.

Background and reasoning

OffsetStore is an append log: each commit encodes (group,TopicPartition,nextOffset) in a LogRecord, and the physical log assigns an increasing offset of its own; the nextOffset stored in the record payload is the consumer progress. key=null and timestamp=0; the provided MessageCodec encodes the OffsetKey/value, and a restart scan finds the latest value for the same key. The underlying store uses the record format and force behavior from Chapter 1, and a damaged tail follows the same recovery rules. consumer.resume first assigns partitions, then looks up the committed value for each one; if no value has been committed, it starts from 0. A consumer offset is the next offset, so after committing 3, recovery resumes by fetching from 3. The broker handler routes COMMIT_OFFSET/FETCH_OFFSET requests as well.

Starting point and edit location.

Already provided: Offset-entry encoding in MessageCodec, RpcClient, consumer assignment, and log-record utilities are provided.

Implement in: `exercises/src/main/java/io/simplekafka/group/OffsetStore.java`: `commit(OffsetKey,long)`, `fetch(OffsetKey)`; `exercises/src/main/java/io/simplekafka/client/SimpleConsumer.java`: `resume(Collection)`, `commitSync()`; also extend `BrokerHandler` with `COMMIT_OFFSET/FETCH_OFFSET`.

Behavioral contract and invariants. An offset is the index of the next record and can be explicitly moved backward; each group and TopicPartition is independent. Reopening the store after a commit still yields the latest value; a missing value is an empty OptionalLong. If nothing has been committed, resume sets position=0; after a successful fetch, commitSync can commit the positions of all assigned partitions. A group token is used for commit fencing only in Step 20; manual consumption passes null.

Tiered hints (not the answer)

Level 1

First define consumer progress as a value, rather than as a physical log-record index.

Level 2

When scanning the same `OffsetKey`, determine which of multiple commits is the final value; do not assume map order.

Level 3

Test both a restarted store and a rebuilt consumer, verifying disk persistence and position initialization separately.

Observable examples and test points. Read offsets 0, 1, and 2, then commit `nextOffset=3`; close and reopen `OffsetStore`, and `fetch` returns 3; a new consumer resumes from 3, while another group still starts from 0. The physical record offset in the `OffsetStore` log does not have to equal the 3 in the payload. See `Step15Test`.

Common pitfalls and differences from real Kafka

Committing the last processed offset (2) directly causes offset 2 to be read again after restart; confusing a group offset with HW leads to the mistaken belief that a commit makes data visible or invisible. Real Kafka stores offsets in the replicated/compacted internal topic `__consumer_offsets`, whereas this course uses a local, single-coordinator append log and provides no HA.

Commands and progression. Use `./gradlew :stepTest -Pstep=15` to diagnose this step; `./gradlew :test -Pstep=15` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is a consumer offset conventionally the next offset rather than the last processed offset?
2. What does HW control, and what does a group's committed offset control?

5.4 Step 16: At-Least-Once Processing**Goals and prerequisites**

Goal: Establish a safe ordering between processing side effects and offset commits, and explain redelivery.

Before you begin, be able to explain: Understand failure and restart, batch positions, and the boundary of commit atomicity.

Background and reasoning

In this course, after one poll, `ProcessingLoop` sorts by `TopicPartition` and invokes the processor in offset order within each partition. It calls `commitSync` once for all partitions only if every record returned in that round was processed successfully; if the processor throws an exception, the original exception propagates and no partition is committed. The consumer's in-memory position has already advanced with poll, so the failed instance must not be reused or it could skip records that were not committed; close it and rebuild it from the persisted positions. Side effects and offset commits are not one atomic transaction: if a side effect has happened but the commit has not succeeded, the record will be processed again after restart. At-least-once therefore allows duplicates, but should not silently lose records that were not successfully processed.

Starting point and edit location.

Already provided: The `RecordProcessor` callback interface, one-round poll, `commitSync`, and the consumer lifecycle needed for rebuilding are provided.

Implement in: `exercises/src/main/java/io/simplekafka/client/ProcessingLoop.java`
a: `runOnce(int, int)`.

Behavioral contract and invariants. Return the number of successfully processed records. Invoke record processing in stable partition order and offset order; commit only after all records have succeeded. Propagate any processor exception unchanged and commit no partitions in that round. After a failure, the caller should close this consumer and rebuild it from the store; do not treat the position already advanced by poll as persisted progress.

Tiered hints (not the answer)

Level 1

Write the timeline “poll, external side effect, commit, crash,” then find the one window in which a duplicate can occur.

Level 2

For a batch spanning multiple partitions, decide whether committing some partitions would violate this lesson’s contract that all processing must succeed before commit.

Level 3

Restart once after processing the first record fails and once after all processing succeeds; compare the delivery sequences observed.

Observable examples and test points. Processing offset 0 succeeds and processing offset 1 throws an exception; after rebuilding the consumer, it receives 0 and 1 again. After both records are processed successfully and committed, rebuilding again does not redeliver them. Record the first side effect in the test and assert replay of 0/1 and the final offset; see **Step16 Test**.

Common pitfalls and differences from real Kafka

Swallowing an exception from the processor and then committing causes data loss; continuing to use the failed instance skips replay. At-least-once does not mean exactly-once, and the broker does not automatically provide idempotent external side effects. Chapter 8 shows how an application can make a local durable effect idempotent by a stable business event id; it does not make the effect and offset commit one transaction. This course has no transactions and does not present its local `OffsetStore` as a transaction boundary.

Commands and progression. Use `./gradlew :stepTest -Pstep=16` to diagnose this step; `./gradlew :test -Pstep=16` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is a duplicate reasonable if a crash occurs after the side effect succeeds but before the offset is persisted?
2. To avoid continuing to run a failed consumer, is the key state its in-memory position or its persisted offset? Why?

Chapter 6

Consumer Groups, Rebalancing, and Commit Fencing

A consumer group lets multiple consumers cooperate to read a topic: each partition is assigned to one member at a time, and membership changes can change the assignment. Knowing “who reads which partition” is not enough; an old member might continue committing old progress after a rebalance, so a generation token is required for fencing. The course deliberately implements an immediately rebalanced, single-topic model, not the complete Kafka classic protocol or the newer consumer group protocol. The coordinator is trusted local state and does not provide HA.

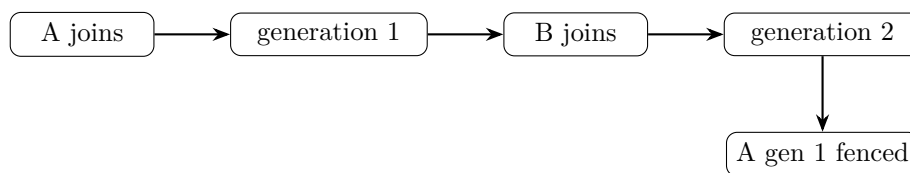


Figure 6.1: A membership topology change increments the generation; an old token cannot overwrite the current commit.

6.1 Step 17: Partition Assignment

Goals and prerequisites

Goal: Implement deterministic, balanced Round Robin assignment with exactly one owner per partition.

Before you begin, be able to explain: Understand set deduplication, stable sorting, and quotient/remainder distribution.

Background and reasoning

If input-set order depends on the network, a HashMap, or startup order, the same group of members may receive an unstable assignment. The course first sorts `TopicPartition` values by topic/partition and members by lexicographic string order, then assigns the i -th partition to `member[i mod n]`. The result is reproducible, and partition counts differ by at most 1 among members. Every known member must appear in the result, even if its partition list is empty; with no members, return an empty map. The shared-topic requirement is checked by the coordinator; the assignor only processes the partitions and members it is given.

Starting point and edit location.

Already provided: The `TopicPartition` value type and group-coordinator dependency interface are provided; the scheduling algorithm is implemented by a single assignor.

Implement in: `exercises/src/main/java/io/simplekafka/group/RoundRobinAssignor.java`: `assign(List<TopicPartition>, List<String>)`.

Behavioral contract and invariants. First deduplicate and sort the inputs in a fixed order. Every distinct partition must appear exactly once, and its owner must be a member; retain members with empty lists. If there are no members, the result is empty. The difference between the largest and smallest number of partitions assigned to any members must be no greater than 1.

Tiered hints (not the answer)

Level 1

Express the unique-owner property as two checkable conditions: complete coverage and no duplicates.

Level 2

First make the member order explicit; the original order of the input `List` is not the course's stable order.

Level 3

Work out five partitions with two members and five partitions with seven members by hand, then test that duplicate input does not assign a partition more than once.

Observable examples and test points. For five sorted partitions and members A and B, A gets three and B gets two; with seven members and five partitions, five members get one each and two have empty lists. Shuffling input order still produces the same result. See `Step17Test`.

Common pitfalls and differences from real Kafka

Returning only members that have partitions loses explicit empty assignments; assigning by the incidental order of a map makes behavior hard to test and reproduce. Real Kafka has multiple assignors, static membership, cooperative rebalancing, and more; this course implements only a simple, single-topic Round Robin and it must not be equated with Kafka's default or complete protocol.

Commands and progression. Use `./gradlew :stepTest -Pstep=17` to diagnose this step; `./gradlew :test -Pstep=17` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is stable sorting part of observable behavior rather than mere output polish?
2. When there are more members than partitions, how do empty assignments help the caller express group state?

6.2 Step 18: Membership and Rebalancing

Goals and prerequisites

Goal: Manage the group membership set and generation, and update all assignments when the topology changes.

Before you begin, be able to explain: Understand that a generation is neither a timestamp nor a consumer offset.

Background and reasoning

Members of a group subscribe to the same topic. When a member joins or an existing member leaves, the assignment changes and the generation increments; a group starts at generation 0. Repeating join for the same member and topic only refreshes its heartbeat and does not create a new generation; changing the subscribed topic is illegal in this simplified model. Each change requires recalculating the complete assignment from the new member set; do not temporarily assign partitions only to the joiner while forgetting to revoke partitions from other members. When the group becomes empty, retain the generation and committed offsets so that later group state is not confused with consumer progress.

Starting point and edit location.

Already provided: The group lock, member-state container, `TimeSource`, assignor interface, and generation lifecycle are provided by the constructor.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `join(String,String,String)`, `leave(String,String)`, `assignment(String,String)`; extend the handler with the corresponding API.

Behavioral contract and invariants. The first join has `generation=1`; a real topology change increments it exactly once and returns the latest assignment for the entire group. Repeating an identical join does not increment it; a different topic reports `INVALID_REQUEST`. `leave/assignment` for an unknown member reports `UNKNOWN_MEMBER`. Retain state and offsets after the group becomes empty.

Tiered hints (not the answer)

Level 1

First identify events that change the member set, then distinguish repeated requests from new state.

Level 2

After rebalancing, every member still in the group must see the full mapping for the same generation.

Level 3

Manually list generations and owners for the sequence A joins, B joins, A leaves, then check a subsequent join after the group is empty.

Observable examples and test points. A's first join returns generation 1; after B joins, generation is 2 and both members share all partitions; after A leaves, it is 3 and B gets every partition. Repeating A's join for the same topic does not increment the generation. See `Step18Test`; `JOIN`, `ASSIGNMENT`, and `LEAVE` are all checked through the real TCP broker API.

Common pitfalls and differences from real Kafka

Advancing each member's assignment version separately can make group tokens disagree; destroying the offset store when the group becomes empty loses restart progress. Real Kafka has a complete coordination protocol with join, assignment synchronization, cooperative revocation, and other phases; the course reassigns immediately and does not simulate the full classic barrier or KIP-848.

Commands and progression. Use `./gradlew :stepTest -Pstep=18` to diagnose this step; `./gradlew :test -Pstep=18` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must members who did not leave also receive a new assignment when the group generation changes?
2. Why can consumer progress be retained when a group has no members?

6.3 Step 19: Heartbeats and Expiration

Goals and prerequisites

Goal: Detect member timeouts under a controllable clock, and atomically process all expired members of the same group as one topology change.

Before you begin, be able to explain: Understand heartbeat time and the expiration threshold.

Background and reasoning

Each member stores `lastHeartbeat`, which is refreshed by a successful heartbeat. A member expires when `now-lastHeartbeat >= sessionTimeoutMillis`. Time comes from the injected `TimeSource`, so tests do not need real sleep. An expiration call may find multiple expired members at once; a single expire call must remove all expired members in the same group together, increment the generation only once, and rebalance only once, avoiding meaningless multiple generations from a single heartbeat tick. Check for an unknown-member error before refreshing; an old-generation error must not extend a member's lifetime either. The provided single-broker and replicated harnesses run periodic expiry workers; the replicated harness also exposes `expireGroups()` for deterministic tests.

Starting point and edit location.

Already provided: The injected `TimeSource`, group lock, session timeout, and scheduling lifecycle are provided; tests can use a deterministic clock.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `heartbeat(GroupToken), expire()`; `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: the `HEARTBEAT` branch.

Behavioral contract and invariants. The boundary uses `>=`. A heartbeat with a valid current token updates the time; an unknown member reports `UNKNOWN_MEMBER`, and an expired generation reports `ILLEGAL_GENERATION`; failures do not refresh the time. `expire` returns the removed `group/member` identifiers; expiration of multiple members in the same group increments the generation only once. Leader/partition concepts are not part of this state.

Tiered hints (not the answer)

Level 1

Keep clock reads, timeout checks, and member removals within a consistent state observation.

Level 2

The return value should distinguish exactly which member was removed from which group, not just return a count.

Level 3

Test the `timeout=100` boundary at times 1099 and 1100, and refresh B before comparing it with A.

Observable examples and test points. With initial time 1000 and `timeout=100`, A remains at time 1099 and is removed at 1100; after B heartbeats at 1099, B still remains at 1100. If A and C in the same group expire at the same time, the generation increments only once. See `Step19Test`; heartbeat uses real TCP, boundary tests use the deterministic hook, and a separate replicated-cluster case verifies worker-driven expiry without heartbeats.

Common pitfalls and differences from real Kafka

Using real system time and sleep makes exact-boundary tests sensitive to scheduler jitter; use an injected clock and explicit expiry hook for deterministic assertions, and keep a separate integration check for the periodic worker. Changing `lastHeartbeat` before validating the token lets an old member renew its lifetime illegally. Real Kafka's session timeout and heartbeat behavior is more complex due to the protocol, consumer configuration, and network scheduling.

Commands and progression. Use `./gradlew :stepTest -Pstep=19` to diagnose this step; `./gradlew :test -Pstep=19` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must the equality boundary be part of the expiration contract?
2. Why should multiple members in the same group expiring in one expire call not cause multiple generation increments?

6.4 Step 20: Consumer Group Fencing

Goals and prerequisites

Goal: Combine group assignment and offset operations, using a generation token to reject writes from zombie members.

Before you begin, be able to explain: Understand the token's group/member/generation triple and revoked partitions.

Background and reasoning

A `GroupToken` consists of group, member, and generation. When `FETCH` or `COMMIT` carries a token, the coordinator must validate that the member exists, the generation is current, and the partition is currently assigned to that member. A consumer with an old generation cannot commit; validation and `OffsetStore.commit` must be in one atomic critical section under the coordinator lock, with lock order coordinator then store and no partition lock held. `GroupConsumer` heartbeats before each poll. On `ILLEGAL_GENERATION`, it queries the current assignment, discards old positions for revoked partitions, resumes new partitions from committed offsets, and fetches with the new token; `UNKNOWN_MEMBER` does not trigger an automatic rejoin and requires an explicit subscribe. A manual token=`null` preserves manual mode and does not pretend to be a group member.

Starting point and edit location.

Already provided: The group messages, RpcClient, SimpleConsumer pull/offset path, and member-lifecycle skeleton are provided; broker requests with tokens are already encoded.

Implement in: `exercises/src/main/java/io/simplekafka/group/GroupCoordinator.java`: `validate(GroupToken, TopicPartition)`; `exercises/src/main/java/io/simplekafka/client/GroupConsumer.java`: `subscribe(String)`, `poll(int, int)`, `commitSync()`; synchronously add the handler fencing.

Behavioral contract and invariants. A token outside the current generation reports `ILLEGAL_GENERATION`; an unknown member reports `UNKNOWN_MEMBER`; an unassigned partition reports `NOT_ASSIGNED`. Failure does not modify the store. Once a new assignment takes effect, an old position for a revoked partition cannot be committed. Commit validation and append complete within one coordinator-lock scope. The server does not guarantee revocation of cached records already sent to an old member; it guarantees only commit fencing.

Tiered hints (not the answer)**Level 1**

Request validation must check token identity, current generation, and partition owner; comparing only the group name is insufficient.

Level 2

After expiration, which old positions can the consumer retain, and which must it rebuild from committed offsets?

Level 3

Create two real TCP consumers, have the old member commit a larger offset with its old token, then check that the store is unchanged after the failure.

Observable examples and test points. A joins at generation 1, then B joins and causes generation 2; A's commit of 100 with the old token is rejected, while B commits 7 with a valid token and the store remains at 7. Two real consumers split four partitions; after one member expires, the other takes over and continues from the committed offset. See `Step20Test`.

Common pitfalls and differences from real Kafka

If the lock is released after validation but before commit, a rebalance can occur in between and a zombie commit can overwrite valid progress. Treating both `ILLEGAL_GENERATION` and `UNKNOWN_MEMBER` as automatic rejoin hides identity errors. Real Kafka group protocols include synchronization barriers and multiple phases, and transactional commit semantics have their own constraints; the course's single-topic coordinator, shared-JVM authority, and local `OffsetStore` are not a highly available `__consumer_offsets`.

Commands and progression. Use `./gradlew :stepTest -Pstep=20` to diagnose this step; `./gradlew :test -Pstep=20` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. What does generation fencing guarantee, and why can it not guarantee exactly-once external side effects?
2. What race can occur if validation and offset append acquire their locks separately?

Chapter 7

Replication, ISR, and High Watermarks

This chapter moves from a single-machine log to replication. **First, read the boundaries of the course’s distributed model carefully:** three independent brokers each have a TCP server, threads, and a disk directory; a follower must fetch records over TCP, and cannot replicate by sharing memory. However, they run in the same JVM and are managed by one trusted ClusterAuthority. The authority linearizes metadata, epoch, and HW state, making teaching failovers controllable. It is not KRaft, has no controller consensus or authority HA, and makes no guarantee about cluster recovery after loss of the entire JVM/authority.

Replication is meaningful only when append, reporting, and visibility boundaries are consistent with one another. LEO is the next offset in each individual log. ISR is the set of replicas currently allowed to participate in acknowledgments. HW is the committed boundary readable by ordinary consumers; consumers can read only offsets $< \text{HW}$. Replicas fetch over a real network; the authority provides consistent state and role decisions, but does not move data.

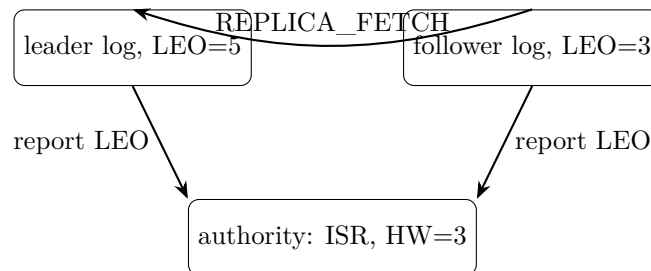


Figure 7.1: Replica logs are fetched over TCP, while progress is serialized by a trusted authority. In this example, $\text{HW} = \min(5, 3) = 3$.

7.1 Step 21: Fetch-Based Replication

Goals and prerequisites

Goal: Have followers persist the leader’s records locally through `REPLICATION_FETCH` requests.

Before you begin, be able to explain: Understand leader LEO, follower LEO, and why a replica fetch differs from a consumer fetch.

Background and reasoning

In the first state-monitor critical section, `FollowerReplicator` verifies that it is online and a follower and captures the epoch; it then performs `REPLICA_FETCH` entirely outside the lock. The leader returns records, epoch, HW, and leader LEO; replication can read up to the current leader LEO and is not restricted by the ordinary-consumer HW. The follower checks the response epoch, first offset, continuity, and alignment with the append offset, then enters a second short critical section, rechecks the epoch, rechecks that it is online and a follower, and performs either an empty poll or a log append within that same critical section. The provided `ReplicaState.update` publishes the role synchronously, preventing a role switch between the final check and append; the network call does not hold the state monitor. Only after the local append is persisted can the next poll report progress. Step 21's network path reads and writes logs only; it does not call the not-yet-implemented report.

Starting point and edit location.

Already provided: The three-node TCP harness, `REPLICA_FETCH` message type, `ReplicaState`, and `PartitionLog` lifecycle are provided; the fixture seed places the leader's actual LEO in the initial progress.

Implement in: `exercises/src/main/java/io/simplekafka/replication/FollowerReplicator.java`: `pollOnce(int, int)`; also add the `REPLICA_FETCH` branch to `BrokerHandler`.

Behavioral contract and invariants. Only the current online follower can replicate the correct partition. Capture the epoch before sending the request; after receiving the response, validate its format outside the network lock, then recheck the current epoch and role inside the local state monitor. An epoch change in the meantime reports `FENCED_EPOCH`; a role change to leader/offline reports `NOT_LEADER`; neither failure may append. The broker conservatively caps oversized positive `maxBytes` to a single-frame reply budget; bounded batches preserve contiguous follower progress across polls. Repeated empty polls do not rewrite data. Step 21 verifies replica data consistency only; it does not report progress or advance HW.

Tiered hints (not the answer)

Level 1

Observe “network response received” and “local persistence completed” as two separate phases.

Level 2

The response contains several boundary values; understand the different purposes of HW and leader LEO.

Level 3

Poll the follower repeatedly, provide an incorrect response epoch, and provide a mismatched first offset; confirm that only a valid batch changes the local LEO.

Observable examples and test points. The leader contains offsets 0–2 and the follower starts at 0; after one poll, reading the follower's disk returns the same key/value pairs and offsets 0, 1, and 2. A second empty poll returns 0 records and leaves LEO unchanged. An incorrect epoch cannot change the follower. Large replies are paged within the frame budget and copied in successive polls. See the real TCP and file assertions in `Step21Test`.

Common pitfalls and differences from real Kafka

Sharing the leader's `PartitionLog` object directly bypasses TCP and local persistence, so the test no longer evaluates replication. Treating HW as the leader LEO prevents a follower from catching up; treating received bytes as success ignores local write failures. Real Kafka has batched

replication requests and more epoch/state interactions; the course’s custom `REPLICA_FETCH` is a teaching API.

Commands and progression. Use `./gradlew :stepTest -Pstep=21` to diagnose this step; `./gradlew :test -Pstep=21` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can a follower replicate records beyond the visibility boundary for ordinary consumers?
2. Why must the follower not report a new LEO before its local append completes?

7.2 Step 22: ISR and High Watermark

Goals and prerequisites

Goal: Maintain ISR and HW from replica progress reports, and remove replicas that lag for too long under a controlled clock.

Before you begin, be able to explain: Understand minISR, monotonic LEOs, and the high-watermark formula.

Background and reasoning

Each replica’s progress is protected by a per-partition lock. Only an assigned/online replica with a valid current epoch may report a nonnegative LEO. A report from the leader advances leaderLEO; a follower’s report must not exceed the leader’s current LEO and must not move backward within the same epoch. A follower refreshes its caught-up time only when it reaches the leader LEO observed at that time; being connected and online alone does not prove synchronization. An expired replica is removed from ISR, but the leader is not. If ISR contains the leader and has at least minISR members, the candidate HW is the minimum of the most recent successful LEO reports in ISR; HW can only advance monotonically. If the minISR condition is not met, HW freezes. Real replication polls are connected to reporting and HW advancement starting in Step 22.

Starting point and edit location.

Already provided: ClusterAuthority provides stable per-TopicPartition locks/Conditions, on-line status, and progress containers; TimeSource is controllable.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicationTracker.java`: `report(int,int,long)`, `expireLagging()`, `advanceHighWatermark()`.

Behavioral contract and invariants. Accept reports only from assigned/online replicas at the current epoch with nonnegative LEO; an invalid report must not modify state. A follower’s LEO must be $\leq$ leaderLEO and must not move backward within the same epoch. If minISR is not met, HW freezes; otherwise $\text{HW} = \max(\text{oldHW}, \min(\text{ISR progress}))$. Wake waiters when report or expiration changes a visible condition. The injected clock defines the time boundary, and the leader is not an expiring replica.

Tiered hints (not the answer)

Level 1

First list the two boundaries each follower report must satisfy: “not greater than” and “not moving backward.”

Level 2

Calculate candidate HW with both sufficient and insufficient ISR, noting that the high watermark cannot move backward.

Level 3

For one set of progress values, observe a catch-up, one expiration, and a minISR decrease, including reports, ISR, HW, and waiter wake-ups.

Observable examples and test points. With leader/follower LEO=(5,3,4) and minISR=2, HW=3; after one follower reports 5, HW=5. If only the leader remains and it appends through 6, minISR=2 is not satisfied and HW remains at 5. A follower whose lag exceeds the lag timeout is removed from ISR; the leader is not. See the controllable-clock checks in Step22Test.

Common pitfalls and differences from real Kafka

Treating every online replica as ISR counts lagging replicas toward acknowledgment; calculating HW from a local LEO that has not been successfully reported can expose unverified data to readers. This course’s authority retains the committed prefix and requires a clean candidate not to fall below the known HW during a controlled failover; that is an additional teaching assumption. Real Kafka does not guarantee the global monotonicity across epochs described here.

Commands and progression. Use `./gradlew :stepTest -Pstep=22` to diagnose this step; `./gradlew :test -Pstep=22` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can appended data not advance HW when minISR is not met?
2. Why must a follower that is online but has not caught up to leaderLEO not refresh its caught-up time?

7.3 Step 23: Produce Acknowledgments**Goals and prerequisites**

Goal: Connect producer acknowledgment semantics to replication state and deadlines, distinguishing “leader has written” from “replicas have committed.”

Before you begin, be able to explain: Understand Condition waiting, epoch fencing, and timeouts that do not roll back appends.

Background and reasoning

ACKS supports only LEADER=1 and ALL=-1. LEADER can return after the leader log append completes, although trailing data may be lost during a failover. ALL must first verify $ISR \geq minISR$, then append and wait for HW to reach `batch.nextOffset`; while waiting, minISR may decrease or the leader epoch may change. The Condition belongs to the authority partition lock; `awaitNanos` releases the lock so replication reports can proceed, and on waking the code rechecks epoch, ISR, HW, and deadline. The deadline uses the monotonic clock `System.nanoTime`; a timeout does not roll back an append. All socket network calls are outside the partition lock.

Starting point and edit location.

Already provided: ReplicationTracker is bound to the authority condition and provides snapshots/locks; RecordData, AppendResult, Acks, and TimeSource are provided.

Implement in: `exercises/src/main/java/io/simplekafka/replication/AckPolicy.java`
 a: `validateBeforeAppend(Acks), await(AppendResult, Acks, int, long)`.

Behavioral contract and invariants. LEADER may return as soon as the append completes. Before append, ALL reports NOT_ENOUGH_REPLICAS if ISR is insufficient; after append, it waits for $HW \geq nextOffset$. If the epoch changes while waiting, report FENCED_EPOCH; if ISR falls below minISR, report NOT_ENOUGH_REPLICAS; if the deadline expires, report REQUEST_TIMEOUT. A timeout does not roll back the written log, and the deadline duration must be nonnegative.

Tiered hints (not the answer)**Level 1**

Distinguish the feasibility check before append from the wait condition after append; they occur at different times and have different errors.

Level 2

A Condition wake-up only indicates that “the state may have changed”; it does not mean the condition has been satisfied.

Level 3

Use latches to control the order of replica reports: make an ALL request wait first, then unblock it with a report; separately verify the difference between LEO and HW after a timeout.

Observable examples and test points. An ALL request remains incomplete after append while a follower has not reported; it succeeds once a report advances HW to nextOffset. A wait timeout leaves the leader LEO advanced, but consumers still cannot see records above HW; a LEADER request can succeed while its record is not yet visible. See the thread-coordination test in Step23Test.

Common pitfalls and differences from real Kafka

Waiting for replication while holding the lock can block the only progress entry point and cause deadlock; resending a timed-out request can create duplicate records; using a wall clock for deadlines is affected by clock adjustments. Real Kafka has more complex acks/ISR configuration and request queues; the course supports only two ACK modes and does not retry automatically.

Commands and progression. Use `./gradlew :stepTest -Pstep=23` to diagnose this step; `./gradlew :test -Pstep=23` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must epoch and ISR be checked again after awaitNanos releases the lock?
2. After an ALL timeout, what is the relationship among LEO, HW, and whether the caller knows an append occurred?

7.4 Step 24: Concurrent Replicated Broker

Goals and prerequisites

Goal: Combine role checks, appends, replication acknowledgments, and HW-truncated fetches in a broker that serves requests concurrently.

Before you begin, be able to explain: Understand lock ordering, waiting across sockets, and waking waiters on close.

Background and reasoning

ReplicatedPartition connects authority state to local logs as a business operation. Under the authority partition lock, produce verifies leader/epoch, appends, updates its own progress, and advances HW; while ALL is waiting, it releases the partition lock so another TCP worker can report follower progress. fetch can read only the complete prefix below HW: an offset $\geq$ HW within the valid range returns empty, while an offset beyond LEO is out of range. Network fetches for replication must also run outside the lock; do not wait on another broker's socket while holding the authority lock. Lock order is fixed as authority lock $\rightarrow$ local log lock; do not acquire locks in the reverse order or nest locks for different partitions. Closing sockets/schedulers must wake ACK waiters and complete them with an error.

Starting point and edit location.

Already provided: The replicated broker's independent endpoints, broker workers, ReplicaState, ReplicationTracker, AckPolicy, and PartitionLog are provided; the single-machine handler continues to be reused.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`: `produce(List<RecordData>,Acks,int,long)`, `fetch(long,int,int,int)`; BrokerHandler explicitly routes to the replicated backend.

Behavioral contract and invariants. produce preserves the linearized order “role and epoch validation $\rightarrow$ pre-append ACK check $\rightarrow$ log append $\rightarrow$ leader report/HW update $\rightarrow$ acknowledgment wait”; it must not hold the partition lock across network calls. fetch returns records at offsets $<$ HW and truncates to the total budget; an offset in $[\text{logStart}, \text{LEO}]$ that is also $\geq$ HW returns empty, while offset $>$ LEO is out of range. An epoch mismatch is fenced. The single-machine and replicated backends are selected explicitly, without falling back by catching an exception.

Tiered hints (not the answer)

Level 1

Draw the locks acquired by the produce, follower-poll, and fetch threads, marking the network boundaries.

Level 2

Check that the HW limit takes effect in fetch before the broker returns results to an ordinary consumer.

Level 3

Pause one follower's replication, keep another socket fetching, and let an ALL produce wait; then resume replication and observe that all three operations can make progress.

Observable examples and test points. Pause follower replication; while ALL produce waits, a fetch over another connection can see only the committed prefix. After replication resumes and reports progress, produce succeeds and the new batch becomes visible. When the broker closes, pending requests complete with errors and the JVM has no worker left blocked. See the real TCP concurrency acceptance test in `Step24Test`.

Common pitfalls and differences from real Kafka

Holding a lock across a network call can prevent the replication thread from acquiring it, and a single-threaded test may not expose this. Sharing HW with the committed group offset confuses replication visibility with consumer progress. Real Kafka runs multiple processes and includes complex controller behavior during network partitions; the independent networks/disks here still share one authority in the same JVM and do not demonstrate production HA.

Commands and progression. Use `./gradlew :stepTest -Pstep=24` to diagnose this step; `./gradlew :test -Pstep=24` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why can another fetch safely read the old HW prefix concurrently while ALL is waiting?
2. When closing, is closing only the socket enough to guarantee that threads waiting on a Condition will finish?

Chapter 8

Election, Log Repair, and Replica Readmission

This chapter connects replica state changes into a controlled failover. Keep the model's limitations in mind: the course's authority is a trusted metadata/HW control plane in a single JVM, and leader changes are triggered explicitly by the course; this is not KRaft controller consensus and does not cover dual-leader arbitration during network partitions. Followers read leader data over real TCP, and each broker still writes its own independent log. The course chooses a clean candidate rather than trading data loss for availability; if there is no eligible candidate, the service must remain unavailable.

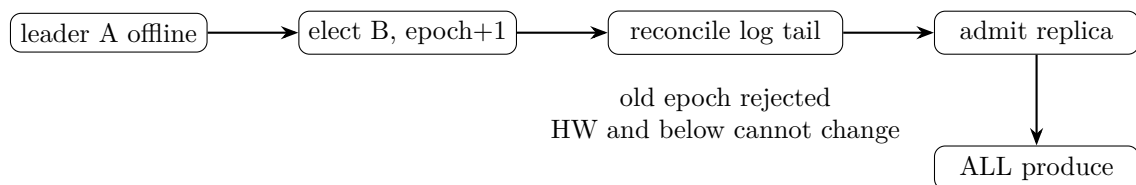


Figure 8.1: A role change must first fence the old role, then repair and prove the replica complete, and only then readmit it to ISR.

8.1 Step 25: Clean Election and Leader Epochs

Goals and prerequisites

Goal: Choose a candidate with the committed prefix when the old leader is offline, and invalidate requests from the old epoch.

Before you begin, be able to explain: Understand the ISR candidate set, HW lower bound, and epoch role fencing.

Background and reasoning

Election is a role decision, not simply a choice of any online node. If the current leader is still online, return the current metadata without incrementing the epoch. Otherwise, choose only from the old ISR among brokers that are online and have $LEO \geq \text{authority.HW}$, using the smallest broker id; a replica below the known commit boundary cannot be a clean candidate. If there is no candidate, report `NO_ELIGIBLE_LEADER`, do not change epoch/HW, and do not perform an unclean election. A successful role change increments the epoch, fences the old leader, makes the new leader the only initial ISR member, and puts the other members in recovering state; they

must be repaired and admitted before they can participate in acknowledgments again. Wake old-epoch producer requests that are waiting and fail them.

Starting point and edit location.

Already provided: The authority owns broker online state, partition locks, LEO/HW, and role publication; reuse the three brokers' existing transports and logs.

Implement in: `exercises/src/main/java/io/simplekafka/replication/LeaderElection.java`: `elect(TopicPartition)`, `checkLeader(int, TopicPartition, int)`.

Behavioral contract and invariants. If the current leader is online, return metadata and epoch unchanged. Switch only to the smallest broker id in the old ISR that is online and has $LEO \geq HW$; on success, increment epoch exactly once, do not roll HW back, and initialize the new ISR with only the leader. Failure for lack of candidates does not change state. A request with an old epoch reports `FENCED_EPOCH`; a request with the correct epoch from a broker that is not leader reports `NOT_LEADER`.

Tiered hints (not the answer)

Level 1

To list eligible candidates, first apply the ISR restriction, then check online status and LEO/HW.

Level 2

Check “epoch is correct” and “requester is leader” separately; they represent different reasons for rejection.

Level 3

Construct three replicas: the old leader is offline, one ISR candidate is below HW, and one online replica is not in ISR; verify that only a candidate satisfying every condition can be elected.

Observable examples and test points. A is the epoch 0 leader, and $HW=3$; after A stops, the smallest broker B in the old ISR with $LEO \geq 3$ is elected at epoch 1, and offsets 0–2 are readable. Produce/fetch requests with the old epoch are fenced. If every member of the old ISR is offline, do not elect C even if it is online, and leave $HW/epoch$ unchanged. See `Step25Test`.

Common pitfalls and differences from real Kafka

Choosing only the most advanced online replica can introduce a non-ISR or unknown divergent log into committed history; quietly choosing a lagging replica when there is no candidate is an unclean election. Real Kafka handles elections through a controller, leader epochs, and cluster state; this course's strong-authority assumption neither covers recovery after losing the authority nor proves consistency across network partitions.

Commands and progression. Use `./gradlew :stepTest -Pstep=25` to diagnose this step; `./gradlew :test -Pstep=25` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. What minimum safety condition does candidate $LEO \geq HW$ guarantee?
2. Why can a non-leader with the correct epoch still not accept produce requests?

8.2 Step 26: Divergent-Log Reconciliation

Goals and prerequisites

Goal: Find the common prefix between an old replica and the new leader, truncate an uncommitted divergence, and catch up to the leader.

Before you begin, be able to explain: Understand HW protection, recovery reads, epoch changes, and the scope of a proof.

Background and reasoning

The course’s repair uses a full-prefix scan as a teaching simplification: from offset 0 to the captured leader LEO, fetch batches using the `recoveryRead=true` flag in `REPLICA_FETCH`, and compare every locally existing record with the corresponding leader record. Recovery reads can see the leader’s tail above HW, but do not report replication progress or modify ISR/HW. If an existing local record at an offset $<$ captured HW differs, report `CORRUPT_RECORD` without changing the file; a merely missing local record can be filled in. Before scanning, capture the current `PartitionLog` object’s `mutationVersion`; after reading the complete local prefix, recheck the epoch, start, LEO, and version under the authority lock, and hold the local log monitor in the same lock order through repair and proof publication. If any modification occurs through the log API during the scan (including truncating and then appending again to reach the same LEO), report `REQUEST_TIMEOUT`; do not use the changed log to make a decision, truncate it, or publish a proof. Only after confirming that the state is stable may divergence be repaired starting at HW and leader contents appended; network reads remain outside the authority lock throughout. The successful proof is bound to the `PartitionLog` object identity and final `mutationVersion`, so it cannot be misused after close/reopen resets the version; this version is an in-memory generation for a single open object, not a durable checksum, and it does not detect direct disk tampering that bypasses the API. The full scan costs $O(n)$ and is used only for explicit recovery.

Starting point and edit location.

Already provided: `PartitionLog`’s `truncateTo/append`, `RpcClient`’s replica fetch, authority epoch/HW snapshots, and partition locks are provided; an old replica is not automatically treated as ISR.

Implement in: `exercises/src/main/java/io/simplekafka/replication/ReplicaReconciler.java`: `reconcile(int expectedEpoch)`.

Behavioral contract and invariants. If the expected epoch is no longer current, exit with `FENCED_EPOCH`. If an existing local record at an offset below the captured HW differs from the leader, report corruption and do not modify the file; a missing committed prefix may be repaired. If the local `mutationVersion` changes during the scan, exit with `REQUEST_TIMEOUT` and do not repair the log. Final revalidation, repair, and proof publication must be atomic under the authority partition lock and the established local log monitor lock order; no truncation point may be below HW. Return LEO on completion and create a proof bound to the captured epoch, leader LEO, local log object identity, and final version; admission must revalidate the proof. Do not report ISR progress during recovery.

Tiered hints (not the answer)

Level 1

When comparing data, distinguish “a local record is missing” from “a local record exists but differs.”

Level 2

Identify which positions are protected by the captured HW and which uncommitted suffix may change.

Level 3

Construct three cases: a committed common prefix, a divergent uncommitted tail, and a lagging empty replica; after repair, verify the disk contents at every offset.

Observable examples and test points. Offsets 0–2 are committed and A has X@3; after switching to B, B writes Y@3. After A reconciles, offsets 0–2 are unchanged, offset 3 contains Y, and LEO=4. An empty replica can also copy the complete committed prefix. If offset 1 differs on the two replicas and captured HW=3, reject the operation and leave A’s file unchanged. See `Step26Test`.

Common pitfalls and differences from real Kafka

Mechanically treating a local HW=0 as “safe to delete all data” loses the committed prefix that was actually replicated; treating a missing prefix as divergent corruption prevents normal catch-up. Real Kafka uses leader-epoch history to locate divergence and truncate precisely; KIP-101 explains why the high watermark alone is insufficient to infer a truncation point. The course’s $O(n)$ scan is for teaching and is not a production approach.

Commands and progression. Use `./gradlew :stepTest -Pstep=26` to diagnose this step; `./gradlew :test -Pstep=26` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why should a different committed record and a missing local committed record be handled differently?
2. If the leader writes again after the captured snapshot, why can the old proof not be used directly for admission?

8.3 Step 27: Metadata Routing Refresh**Goals and prerequisites**

Goal: Let clients cache metadata and route by partition leader, while avoiding automatic replay of a business request after an error.

Before you begin, be able to explain: Understand control-plane queries, data-plane routing, and uncertain acknowledgments.

Background and reasoning

A bootstrap endpoint is only an entry point for discovery; it does not have to serve data permanently. refresh queries metadata in bootstrap order and can continue to another available node if a connection fails. call sends one business request to the leader/epoch in the cache. On receiving `NOT_LEADER` or `FENCED_EPOCH`, it may refresh the cache so the next request uses the new role, but it must propagate the original error from the original request to the caller. It must not automatically replay the business request using new metadata: the old request’s error may have occurred while waiting for confirmation, after data was already appended. I/O, timeout, and storage errors are not retried automatically either. Producers, consumers, and group consumers

may add a Router constructor entry point; the original `RpcClient` constructor continues to support single-machine mode and is not a deprecated shim. Group/offset control-plane operations use an available bootstrap control client, while partition fetch/produce is routed to the leader.

Starting point and edit location.

Already provided: `RpcClientFactory` can connect by Endpoint; `MessageCodec` and the original client logic are complete; broker metadata returns the actual leader endpoint/epoch.

Implement in: `exercises/src/main/java/io/simplekafka/client/MetadataRouter.java`: `refresh(String)`, `call(TopicPartition, Request)`; connect the Router constructor paths in `SimpleProducer`, `SimpleConsumer`, and `GroupConsumer`.

Behavioral contract and invariants. `refresh` reads metadata only and tolerates failures in bootstrap order; `call` uses the cached endpoint and epoch. `NOT_LEADER/FENCED_EPOCH` may trigger refresh, but the original response must be propagated and `produce/fetch/commit` must not be resent. Other network and storage errors are not retried automatically. Only a later, independent request uses the updated cache.

Tiered hints (not the answer)

Level 1

Make clear that one call makes exactly one business-request attempt; `refresh` is a control-plane action, not a retry.

Level 2

How does the order of failures across different endpoints affect the initial metadata query?

Level 3

Simulate a write rejected using stale metadata even though the server may already have appended it; check the refreshed metadata, the old receipt, and the record count, then issue the next request explicitly.

Observable examples and test points. Old metadata points to A/epoch 0. After A stops, the first routed request receives an error related to the refresh; the client refreshes to B/epoch 1 but returns that error. A second, explicit new request writes only one copy to B. If a broker appends but its response is lost, the client reports an error and does not automatically create a second record. See `Step27Test`.

Common pitfalls and differences from real Kafka

Treating `NOT_LEADER` as proof that “nothing was written” can cause duplicate production; refreshing the cache but returning success to the caller fabricates a receipt. `MetadataRouter` refreshes but never replays a request, and `SimpleProducer` never retries automatically. Chapter 8 adds a separate `IdempotentProducer` with caller-invoked retry and durable batch identity; it does not change the ordinary producer contract or provide transactions or end-to-end exactly-once guarantees.

Commands and progression. Use `./gradlew :stepTest -Pstep=27` to diagnose this step; `./gradlew :test -Pstep=27` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why are refreshing metadata and replaying a business request two different actions?
2. When a client sees an error, why can it not conclude that the server had no side effect?

8.4 Step 28: Replica Readmission to ISR

Goals and prerequisites

Goal: Readmit a repaired replica to ISR only after strict proof, and verify an integrated failover.

Before you begin, be able to explain: Understand replica assignment, online status, follower role, epoch, and proof validity.

Background and reasoning

Completion of replica recovery does not automatically mean that the replica is synchronized. The prefix proof generated by log repair is required to request readmission to ISR, and is bound to the captured epoch, leader LEO, local PartitionLog object identity, and mutationVersion. Under the authority partition lock, verify that the target broker is assigned and online, is currently a follower, and that the expected epoch matches the current epoch. Hold the target PartitionLog monitor until the ISR transition is published; append/delete/truncate/close use the same monitor, so there is no local log mutation window between proof validation and ISR admission. The proof must still match the current leader epoch/LEO and the same log object/version passed in the current call; log start must be 0 and current LEO must exactly equal leader LEO. Any append/delete/truncate through the PartitionLog API invalidates the old proof, even if truncating and appending restores the same LEO; reconciliation must be run again. This in-memory version cannot detect external file changes that bypass the API. On successful ISR admission, initialize reportedLEO/lastCaughtUp and recompute HW; a repeated addition of a member already in ISR returns true but does not change HW.

Starting point and edit location.

Already provided: The authority's role/ISR management, replication progress, proof type, and broker lifecycle are provided by the course's base layer.

```
Implement in: exercises/src/main/java/io/simplekafka/replication/ReplicaAdmission.java
: tryAdd(int, TopicPartition, int).
```

Behavioral contract and invariants. Return true only if the target replica is assigned, online, a follower for the current epoch, and the proof's leader and local log object identity/version are correct; the log start is 0 and local LEO equals the current leader LEO. Do not allow a local PartitionLog API write during proof validation; an append by the leader or any local PartitionLog change makes the old proof return false. An unassigned replica reports INVALID_REQUEST; an offline replica or stale proof returns false. A repeated call for a replica already in ISR returns true without changing the high watermark. On success, recompute HW and wake acknowledgment waiters.

Tiered hints (not the answer)

Level 1

Treat proof validity as a versioned fact, not a permanent Boolean flag.

Level 2

If leader LEO changes after proof generation, which state must be compared again?

Level 3

First verify admission in a simplified case with no changes; then append one leader record and try reusing the old proof; finally retry with a new proof.

Observable examples and test points. Stop A and let B take over; repair and admit surviving C, then restore minISR; write a new message with ALL. Restart A, repair and

admit it, then write again with ALL. The three independent logs contain matching record bytes/offsets without gaps, and the group continues from its committed offset. Reusing an old proof after leader LEO changes must return false. See the full TCP integration acceptance test in `Step28Test`.

Common pitfalls and differences from real Kafka

Checking only equal LEOs without comparing the prefix lets a divergent replica masquerade as synchronized; checking only the epoch at proof creation without checking the current leader LEO can omit new tail records. Real Kafka has more complex ISR admission and leader-epoch/replication protocols; the course's strict prefix proof and strong authority are verifiable teaching constraints, not a production controller design.

Commands and progression. Use `./gradlew :stepTest -Pstep=28` to diagnose this step; `./gradlew :test -Pstep=28` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is matching LEO insufficient to prove that two logs contain the same data?
2. Why must admission recompute HW and wake requests waiting for acknowledgment?

Chapter 9

Consistency Across Producers, Brokers, and Consumers

This chapter follows one message through three independent boundaries: producer confirmation and retry, broker replication and visibility, and consumer progress versus durable business effects. A successful result at one boundary is not an end-to-end transaction. The experiments use real TCP connections and per-broker logs; their observations distinguish physical log end (LEO), the high watermark (HW), consumer position, committed offset, and processor callbacks.

9.1 Step 29: Cross-Layer Consistency Experiments

Goals and prerequisites

Goal: Orchestrate reproducible failures across producer, broker, and consumer boundaries, then report what actually happened at each layer.

Before you begin, be able to explain: Understand the difference between a successful append, a lost acknowledgment, HW visibility, a consumer's in-memory position, a committed group offset, and an external side effect.

Background and reasoning

The supplied `FaultProxy` forwards raw frames to a real broker and can either forward a matching request then drop its reply, or reject a matching request before forwarding it. `EffectRecorder` records every callback, including duplicates. `ConsistencyObservation` captures a named phase, error, replication snapshot, physical LEOs for all brokers, records fetched from the current leader below HW, optional consumer position and group committed offset, observed effects, and optional append receipt. `ConsistencyTrace` groups those observations for one partition. Build each scenario from the caller-owned, already-started `ClusterHarness`: an empty one-partition topic, leader 1, three replicas in ISR, `minISR=2`, `HW=0`, three physical LEOs at 0, no committed group offset, and unstamped input data. Fault scenarios also receive a started, unarmed proxy whose upstream is broker 1. Use the injected fixed clock and the supplied replication helper rather than sleeps or expiry races. Its API is `FaultProxy(endpoint)`, `upstream()`, `start()`, `endpoint()`, `armed()`, `dropNextReply(short)`, `rejectNextRequest(short)`, `forwarded(short)`, `dropped(short)`, `rejected(short)`, and `close()`. Only one API fault can be armed at once. A drop fault forwards the request, reads the real broker reply, then closes the downstream connection without replying; a reject fault does not forward and returns `REQUEST_TIMEOUT` with `ErrorBody("injectedpre-forwardfailure")`. `LabSupport` supplies `replicateAndReport(cluster, tp)`, `observe(phase, error, cluster, tp, position, group, effects, receipt)`,

and `awaitLeaderLeo(cluster, tp, expectedLeo)`. Observation fields, in order, are `phase`, `error`, `replication`, `physicalLeo`, `position`, `committedOffset`, `visible`, `effects`, `receipt`; the visible records come from real leader FETCH below HW and physical LEOs come from each log, not authority reports. `EffectRecorder` implements `RecordProcessor`; `synchronized process(tp, record)` appends every callback and `synchronized snapshot()` returns a stable copy without deduplicating. `ConsistencyObservation` is `recordConsistencyObservation(String phase, ErrorCode error, ReplicationSnapshot replication, Map<Integer, Long> physicalLeo, OptionalLong position, OptionalLong committedOffset, List<LogRecord> visible, List<LogRecord> effects, Optional<AppendResult> receipt)`; `ConsistencyTrace` is `recordConsistencyTrace(TopicPartition tp, List<ConsistencyObservation> observations)`.

Starting point and edit location.

Already provided: `FaultProxy`, `EffectRecorder`, `ConsistencyObservation`, `ConsistencyTrace`, and `LabSupport` provide fault injection, recording, and actual-state observation.

Implement in: `exercises/src/main/java/io/simplekafka/lab/ConsistencyExperiments.java`: `producerOutcomeUnknown(ClusterHarness, TopicPartition, RecordData, FaultProxy)`, `leaderAckLoss(ClusterHarness, TopicPartition, RecordData)`, `allAckTimeout(ClusterHarness, TopicPartition, RecordData)`, `consumerReplay(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder, FaultProxy)`, and `staleGroupSideEffect(ClusterHarness, TopicPartition, String, RecordData, EffectRecorder)`.

Nonmatching APIs pass through unchanged; the proxy counters increase only for the corresponding real forwards, drops, or pre-forward rejections. An invalid API or an attempt to arm a second fault reports `INVALID_REQUEST`. The five editable methods are public static and return `ConsistencyTrace`; `consumerReplay` also declares `throwsException`.

Behavioral contract and invariants. Implement all five methods using caller-supplied data. Require a started cluster with one empty partition, leader 1, ISR 1,2,3, minISR=2, HW=0, all physical LEOs 0, and unstamped input; consumer scenarios also require an uncommitted group and an empty caller-owned recorder. The two fault-injection methods require a started, unarmed proxy whose upstream is broker 1. Invalid starting state reports `INVALID_REQUEST`. Close only scenario-owned clients; record only expected failures and propagate unrelated ones. `producerOutcomeUnknown` drops one successful LEADER reply and reports `UNKNOWN` (LEO 1/0/0, HW 0, position 0, no visible record or receipt), then `POISONED` when reuse of that `SimpleProducer` is rejected without another forwarded `PRODUCE`. An independent ordinary producer explicitly resends the same data and obtains [1,2]; after replication report `EXPLICIT_RESEND` (LEO 2/2/2, HW 2, position 2, visible offsets 0 and 1 contain equal input data, and no group committed offset is observed). `leaderAckLoss` obtains LEADER receipt [0,1] without replication and reports `ACKED_NOT_VISIBLE` (LEO 1/0/0, HW 0, position 0); stop broker 1, elect broker 2 at epoch 1 and report `ELECTED_WITHOUT_TAIL`; restart broker 1, reconcile, and report `OLD_TAIL_REMOVED` (all LEOs 0, HW 0, no visible record). `allAckTimeout` uses ALL with timeout 0 and reports `TIMEOUT_NOT_VISIBLE` (LEO 1/0/0, HW 0, position 0, no receipt); after real replication and reporting, the same consumer reads offset 0 and reports `LATE_VISIBLE` (LEO 1/1/1, HW 1, position 1), without a resend or retroactive success ACK. `consumerReplay` writes and replicates one record to HW=1; the consumer reads offset 0 and the proxy rejects `COMMIT_OFFSET` before forwarding. Report `COMMIT_FAILED` with `REQUEST_TIMEOUT`, position 1, no committed offset, effects [0]. Close that consumer; a new connection resumes at 0, processes and commits again, then reports `REPLAY_COMMITTED` with position and committed offset 1 and effects [0,0]. `staleGroupSideEffect` has member z join, fetch offset 0, and record an effect before member a joins and takes the assignment. z's old token gets `ILLEGAL_GENERATION` for both `COMMIT` and `FETCH`; report `STALE_COMMIT_REJECTED` (position 1, no committed offset, effects [0]). The current owner fetches offset 0, records it, and commits nextOffset=1; report `NEW_OWNER_COMMITTED` (committed offset 1, effects [0,0]). Fencing rejects stale requests but cannot undo an earlier callback.

The UNKNOWN and POISONED observations carry REQUEST_TIMEOUT; the poisoned reuse is rejected locally and adds no proxy forward.

Tiered hints (not the answer)

Level 1

Track the physical LEO of each broker separately from the authority's reported LEO and from HW.

Level 2

Write a timeline for each failure and label which side has already performed its action when the error occurs.

Level 3

Keep the recorder and proxy in the test caller: assert their counters and snapshots independently of the returned trace.

Observable examples and test points. Check every phase against the real cluster and caller-owned recorder/proxy. On response loss, expect one dropped PRODUCE reply and two forwarded PRODUCE requests: the poisoned attempt is not forwarded, and the explicit resend accounts for the second forward. In the election trace, ELECTED_WITHOUT_TAIL is still LEO 1/0/0 at HW 0, then OLD_TAIL_REMOVED is LEO 0/0/0. The ALL failure is REQUEST_TIMEOUT; its later visibility does not retroactively produce a receipt. For consumer replay, assert rejected(COMMIT_OFFSET)=1, forwarded(COMMIT_OFFSET)=0, effects [0,0], and a real FETCH_OFFSET of 1 after the new consumer commits. For stale-group replay, the old token's two requests are fenced but effects remain [0,0].

Common pitfalls and differences from real Kafka

A receipt is evidence of an ACK result, not proof that a LEADER-acknowledged tail will survive a later clean election. A timeout is not proof that no append occurred. A consumer position advances before a group commit, and group fencing protects broker operations rather than reversing a side effect. These experiments do not add retries to SimpleProducer, use sleeps to manufacture timeouts, or claim end-to-end exactly-once processing.

Commands and progression. Use `./gradlew :stepTest -Pstep=29` to diagnose this step; `./gradlew :test -Pstep=29` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Which observations distinguish a LEADER acknowledgment from replication-committed visibility?
2. Why can a rejected stale commit coexist with a side effect that has already happened?

9.2 Step 30: Idempotent Producer

Goals and prerequisites

Goal: Persist producer batch identity with each record and make an explicit retry return the original offset range without appending duplicates.

Before you begin, be able to explain: Understand batch sequence, producer epoch, content fingerprints, partial replication, and why retry behavior must be an explicit client choice.

Background and reasoning

The existing three-argument `RecordData` constructor remains the normal, non-idempotent record path. Its new nullable `producerStamp` component is metadata assigned by the broker for an idempotent request. `ProducerStamp` contains `producerId`, caller-managed `producerEpoch`, `firstSequence`, `batchSize`, `batchIndex`, and a 32-byte `batchHash`. The SHA-256 fingerprint is streamed over count and each record's timestamp, key length/key, and value length/value; it excludes offsets and leader epochs, and distinguishes null from empty keys. The fingerprint verifies that a retry with the same identity carries the same batch. A valid stamp has nonnegative `producerId`, `producerEpoch`, and `firstSequence`; positive `batchSize`; `batchIndex` in $[0, \text{batchSize})$; a non-overflowing `firstSequence+batchSize`; and exactly 32 hash bytes. Its fingerprint input is big-endian `count:int32`, then per record `timestamp:int64|keyLength:int32(-1 for null)|key|valueLength:int32|value`; offsets and leader epochs are excluded.

The supplied `ProducerStamp.ENCODED_OVERHEAD` is 64 bytes; that overhead counts against the 1 MiB record bound in both disk and `MessageCodec` wire budgets. Each stamped record keeps the identity in its ordinary log entry, so `FETCH`, `REPLICA_FETCH`, reconciliation, and disk recovery all see the same metadata. The stamped disk layout is `length|crc32c|offset|timestamp|marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes|keyLength:int32|valueLength:int32|key|value`. It adds 64 bytes, for a minimum total record size of 96 bytes; the 1 MiB record limit includes the stamp, and CRC32C covers offset through value. Ordinary unstamped bytes do not change. On the teaching wire, a stamped `RecordData` uses `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`; ordinary wire records remain unchanged.

The broker derives deduplication state from the retained log prefix, not from a new authority truth. It validates sequences and complete batch hashes before writing, remembers completed batch ranges, and allows a replication fetch to append a matching partial batch suffix. A completed duplicate returns its original range; a retry that completes a partial tail appends only the missing suffix. A higher producer epoch starts at sequence zero and takes effect only with a durable record. A changed hash for the same batch identity conflicts; gaps, unrecorded old sequences, or invalid boundaries are out of order. An incomplete tail blocks unrelated appends until that original batch is completed or normal repair changes the prefix. If retained history starts above zero, producer state is expired rather than guessed.

Starting point and edit location.

Already provided: The stamp value, fingerprint helper, record/request serialization support, and replicated log infrastructure are supplied. Extend the existing storage and routing algorithms as well as implement the new client and replicated-produce path.

Implement in: `exercises/src/main/java/io/simplekafka/client/IdempotentProducer.java`: `send(TopicPartition, List, Ack, long)`, `retry(TopicPartition)`; `exercises/src/main/java/io/simplekafka/replication/IdempotentAppender.java`: `append(long, int, long, List)`, `recover()`; `exercises/src/main/java/io/simplekafka/replication/ReplicatedPartition.java`: `produceIdempotent(List, Ack, int, long, long, int, long)`; `exercises/src/main/java/io/simplekafka/broker/BrokerHandler.java`: `handle(IDEMPOTENT_PRODUCE)`; `exercises/src/main/java/io/simplekafka/client/MetadataRouter.java`: `call`; extend `RecordCodec.encode/decode`, `SegmentLog.append/recover`, and `PartitionLog.append/deleteBefore/truncateTo`.

The supplied partition-log support also exposes `tailProducerStamp()`, `prefixVersion()`, and `truncateToOffset(long)` so append validation and cache invalidation can follow the actual persisted tail and rewritten prefix. The supplied `IdempotentProducer(MetadataRouter, long, int)` constructor and `close()`, and the `IdempotentAppender(PartitionLog)` constructor,

provide lifecycle setup; implement only the listed algorithm methods. `IdempotentAppender` construction does not automatically run `recover`; recovery is an explicit algorithm call. The public algorithms return `ProduceReceipt` from `send(TopicPartition, List<RecordData>, Acks, long)` and `retry(TopicPartition)`, `AppendResult` from `IdempotentAppender.append(long, int, long, List<RecordData>)` and `ReplicatedPartition.produceIdempotent(List<RecordData>, Acks, int, long, long, int, long)`, and `void` from `recover()`. The supplied `IdempotentProduceBackend` capability is implemented by `ReplicatedPartition`; `BrokerHandler` rejects API 11 with `INVALID_REQUEST` when a backend lacks it, including the single-broker backend.

Behavioral contract and invariants. `IdempotentProducer.send` fixes the per-partition sequence and a stable copy of the complete input batch before one RPC. Only a valid successful receipt advances the sequence and clears pending state. Any request/reply failure keeps the batch pending; another send to that partition is rejected, while `retry` explicitly refreshes metadata and makes exactly one request with the same identity, sequence, payload, ACK mode, and timeout. There is no background or automatic retry, and closing the producer discards only local pending state without closing its borrowed router. The caller supplies producer id and epoch; a rebuilt client must use a higher epoch and sequence zero. `IdempotentAppender` validates all inputs before mutation, returns the original range for an exact completed duplicate, appends only the missing suffix of an identical partial batch, and rejects a same-identity/different-hash batch with `DUPLICATE_SEQUENCE_CONFLICT`. Old producer epochs report `FENCED_PRODUCER_EPOCH`; gaps and invalid old sequences report `OUT_OF_ORDER_SEQUENCE`; retained-prefix loss reports `PRODUCER_STATE_EXPIRED`; unrelated writes while a tail batch is incomplete report `INCOMPLETE_BATCH`. After a durable mutation fails or leaves ambiguous state, the log is failed and cannot accept further operations until it is reopened. Prefix truncation or retention invalidates derived cache state; recovery scans and verifies the retained history. The wire API is id 11, `IDEMPOTENT_PRODUCE`, with request fields `tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records`; success uses the existing `ProduceBody`. Reject empty, oversized, invalid, or already stamped client batches without side effects. This API is available only through the replicated backend, not the single-broker backend. Duplicate requests still pass ACK prechecks and the full ACK wait, including ALL's minISR, epoch, HW, and deadline rules.

The log-derived state tracks each producer's highest durable epoch, next sequence, and completed batch identity/range; the partition has at most one incomplete tail batch. An unknown producer or a higher epoch starts at sequence zero; within the current epoch, a new batch must start at exactly the stored next sequence. Recovery reads in bounded batches, verifies batch continuity and hashes, and replays only the new suffix while the prefix version is unchanged. If the prefix changes or LEO rolls back, discard the cache and rebuild from offset zero; a recovery error invalidates the derived state rather than skipping damaged history. Client details: a pending batch blocks only another `send` for the same partition; other partitions are independent. A successful receipt must cover exactly the batch's record count before sequence advancement. `retry` with no pending batch reports `INVALID_REQUEST`; refresh failure and invalid replies preserve pending state, and a retry uses the refreshed current leader epoch. A repeated batch still performs ACK precheck and the full ACK wait.

Tiered hints (not the answer)

Level 1

Keep the three identities separate: leader epoch fences broker roles, producer epoch fences a producer session, and sequence identifies a batch within that producer session.

Level 2

Treat the log as the durable source of truth and the in-memory producer-state cache as a versioned replay result.

Level 3

Test one complete duplicate, one same-sequence changed-content conflict, one partial-batch continuation, and one explicit retry after a leader change; inspect both receipt ranges and stamped disk records.

Observable examples and test points. Send a multi-record batch and repeat the same identity and payload: both receipts name the same half-open range, while physical LEO and stamped bytes do not change on the duplicate. The following sequence appends at the next offset. After the first response is dropped, copy the records to HW, elect a clean new leader, and explicitly retry to obtain the original range with only one copy on every broker. Reopen a broker's log from disk and issue the same raw request: it still returns the original range. A repeated ALL request must still time out while its range is above HW and can succeed only after the range is replicated and acknowledged.

Common pitfalls and differences from real Kafka

This teaching identity is caller-supplied; it does not implement Kafka PID allocation, a transaction coordinator, or client session WAL. A new producer id does not deduplicate a repeated business event, and a business event id is not a producer sequence. `SimpleProducer` remains non-idempotent and never retries; the new `IdempotentProducer` retries only when its caller invokes `retry`. Idempotent append does not strengthen ISR, HW, or election assumptions and does not create an end-to-end exactly-once guarantee.

Commands and progression. Use `./gradlew :stepTest -Pstep=30` to diagnose this step; `./gradlew :test -Pstep=30` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why must a duplicate ALL request still check the current HW and minISR?
2. What information is lost if producer deduplication state cannot be reconstructed from the retained log prefix?

9.3 Step 31: Idempotent Consumer Effects**Goals and prerequisites**

Goal: Apply each stable business event to a durable local ledger once, even when its Kafka record is delivered more than once.

Before you begin, be able to explain: Understand why delivery deduplication and business-event deduplication use different identities, and why external effects are not atomic with consumer offsets.

Background and reasoning

`BusinessEvent` contains a stable `eventId`, an `account`, and a signed `delta`. Both identifiers are non-empty strict UTF-8 strings of at most 255 bytes; delta may be positive, negative, or zero. Its versioned value encoding is `version:uint8=1|eventId:identifier|account:identifier|delta:int64`. The supplied `MessageCodec.encodeBusinessEvent/decodeBusinessEvent` uses the course identifier encoding and rejects malformed or trailing data. The store appends the encoded event id, account, and delta as one ledger record; its in-memory

identity and balance maps are rebuilt by replay rather than persisted separately. The codec signatures are `public static byte[] encodeBusinessEvent(BusinessEvent)` and `public static BusinessEvent decodeBusinessEvent(byte[])`; they reject unknown versions, truncated input, invalid UTF-8 or identifiers, and trailing bytes. For the consumer demo, the supplied processor decodes `record.data().value()` with `MessageCodec.decodeBusinessEvent` and calls `DurableEffectStore.apply`; `ProcessingLoop` keeps its existing process-then-commit order. The supplied `balance(account)` validates the account identifier by the same rule and returns zero when no balance exists.

Starting point and edit location.

Already provided: `BusinessEvent`, event serialization, and `DurableEffectStore(Path)`, `balance(String)`, and `close()` are supplied.

Implement in: `exercises/src/main/java/io/simplekafka/client/DurableEffectStore.java`: implement `apply(BusinessEvent)` and `recover()`.

The supplied store API is `DurableEffectStore(Path)`, `synchronizedbooleanapply(BusinessEvent)`, `synchronizedlongbalance(String)`, `synchronizedvoidrecover()`, and `synchronizedvoidclose()`; `close` is idempotent. The supplied constructor opens the local log and rebuilds store state through `recover()` before use.

Behavioral contract and invariants. Use `<directory>` as a local `PartitionLog`. One event is one `RecordData(null, encodedEvent, 0)` and the record append is forced before publishing in-memory state. On the first valid event id, compute the new account balance with checked addition, append once, then publish the full event in `seen` and the new balance; return true. An identical event id, account, and delta returns false and changes neither log nor balance. Reusing an event id with different content or overflowing a balance reports `INVALID_REQUEST` without appending. `recover` rebuilds both maps in retained offset order and requires `logStart=0`, null keys, timestamp 0, and no producer stamp. It decodes each payload, validates event identities, and counts an identical repeated event only once; malformed complete records, conflicting ids, overflow, or a scan with no progress report `CORRUPT_RECORD`. Synchronize `apply`, `balance`, `recover`, and `close` on the same store instance. If an append fails with an uncertain durable result, publish no new in-memory balance and mark the store failed; later operations report `STORAGE_ERROR`, so the caller must close and reconstruct from disk.

Recovery scans the retained ledger in bounded reads and applies records in offset order; an incomplete physical tail follows the underlying log's recovery rule, while complete malformed records are rejected. A closed or failed store rejects `apply`, `balance`, and `recover` with `STORAGE_ERROR`; `close` remains idempotent.

Tiered hints (not the answer)

Level 1

The event id must come from the business event and stay the same across Kafka offsets and process restarts.

Level 2

Validate the event-id conflict and checked balance before touching the log; publish memory only after the append succeeds.

Level 3

Replay the same event from a new consumer position after closing and reopening the store, then compare both its balance and its ledger records.

Observable examples and test points. Process event A (account +10), a second Kafka record carrying the same A, and event B (account -3). The first A and B each append once; the repeated A returns false, leaving balance 7 and ledger entries [A,B]. If the first commit fails, rebuild the consumer and replay offsets 0, 1, and 2; verify callback history [0,1,2,0,1,2],

the same two ledger entries, balance 7, and committed offset 3. A stale generation can reject the commit but cannot reverse the durable effect.

Common pitfalls and differences from real Kafka

The store makes one local ledger append atomic with its in-memory publication; it does not share a transaction with Kafka or the consumer offset store. A callback can run more than once, including for the same event at different offsets. Group fencing rejects an old token but cannot roll back an effect. This is application-level idempotent accounting for one local store, not a distributed transaction, global ledger, or end-to-end exactly-once claim.

Commands and progression. Use `./gradlew :stepTest -Pstep=31` to diagnose this step; `./gradlew :test -Pstep=31` runs cumulatively from Step 1 through this step. A chapter is complete only when the cumulative tests through its final step (4, 8, 12, 16, 20, 24, 28, or 31) all pass.

Check your understanding

1. Why is a stable event id a better deduplication key than a Kafka offset or a callback count?
2. What remains uncoordinated when the ledger append succeeds but the consumer offset commit fails?

Appendix A

Formats, Error Codes, and Interface Index

This appendix gathers stable conventions used repeatedly across chapters for convenient reference. If anything here differs from a public signature in an exercise, follow the course manifest and Java types. Layouts and error codes defined by this course belong only to the simpleKafka teaching interface; unless an official source is explicitly cited, do not describe them as Kafka wire formats, storage formats, or configuration defaults.

A.1 Disk and Network Formats

A.1.1 Records and Indexes

Records use big-endian encoding:

Field	Width	Semantics
length	int32	Record length excluding itself and including CRC; 28 to 1,048,576.
crc32c	int32	Checks offset through the end of value, excluding the length/CRC fields themselves.
offset	int64	Nonnegative logical offset.
timestamp	int64	Nonnegative timestamp.
keyLength	int32	-1 means null, 0 means a present empty key, otherwise the number of bytes.
valueLength	int32	Nonnegative number of bytes; value is not null.
key/value	variable	Variable-length fields follow in the order described above.

The fixed-length part of each record (excluding the length prefix) contains the CRC, offset, timestamp, and two lengths, for a total of 28 bytes; the length prefix makes the minimum total size 32 bytes. The lower bound for the length field remains 28 because its count starts at the CRC. Each segment's sparse-index entry is `offset:int64,position:int64`, totaling 16 bytes; one index point is written for every intervalRecords records, and the first record always has a point. The index may be discarded and rebuilt from the log. Ordinary records retain this layout. A stamped record inserts the following 64-byte producer identity after timestamp and before keyLength: `marker:int32=-2|producerId:int64|producerEpoch:int32|firstSequence:int64|batchSize:int32|batchIndex:int32|batchHash:32bytes`. Its full layout is `length|crc32c|offset|timestamp|marker|producerId|producerEpoch|firstSequence|b`

atchSize|batchIndex|batchHash|keyLength|valueLength|key|value. The minimum total size is 96 bytes, and the 1 MiB maximum includes the stamp. CRC32C covers offset through the final value byte, including the stamp. Ordinary record bytes are unchanged. The supplied `ProducerStamp.ENCODED_OVERHEAD` is 64 bytes and is included in the 1 MiB record bound for both disk and `MessageCodec` wire records.

The directory convention is `<root>/<topic>/<partition>/<baseOffset:020d>.log`, with the paired index suffix `.index`. The retained range is `[logStartOffset,LEO)`, and fetch is left-closed/right-open. LEO is the next writable offset; HW is the highest committed boundary visible to ordinary consumers (only offsets `< HW` are visible). A group committed offset is also a next offset, but is not the same as HW.

A.1.2 TCP Frames and Request Bodies

The teaching frame is `length:int32|version:int16(1)|api:int16|correlationId:int32|error:int16|payload;` length excludes itself and is in the range `[10,8,388,608]`. EOF before the first byte of a new frame returns null; EOF within a frame reports `INVALID_REQUEST`. The request error field must be 0. Unknown versions/APIs, oversized frames, and trailing body bytes are all rejected. The maximum size of an individual disk record remains 1 MiB and is not changed by the frame limit.

API ids are fixed as follows:

ID	API	ID	API
1	METADATA	6	JOIN_GROUP
2	PRODUCE	7	HEARTBEAT
3	FETCH	8	LEAVE_GROUP
4	COMMIT_OFFSET	9	GROUP_ASSIGNMENT
5	FETCH_OFFSET	10	REPLICA_FETCH
11	IDEMPOTENT_PRODUCE		

`MessageCodec` uses big-endian primitive fields; strings are an int32 UTF-8 byte length followed by bytes, collections are an int32 count followed by each item, Optional values use a boolean followed by the value, and `byte[]` uses an int32 length (only key may be -1). Request and successful-response fields are encoded in the declaration order of the provided Messages nested records; map results are sorted by key. When the reply header has `error=NONE`, the body is the corresponding success record; on error, the body is `ErrorBody(message)`, and the error code appears only in the frame header. Field schemas are fixed by `provided/.../protocol/Messages.java`; students do not need to redefine the eleven message formats. A stamped `RecordData` on `FETCH` or `REPLICA_FETCH` wire uses `marker=-2|producerId|producerEpoch|firstSequence|batchSize|batchIndex|batchHash[32]|keyLength|key|valueLength|value|timestamp`; an ordinary record's wire encoding is unchanged.

A.1.3 Message Body Field Order

The requests and responses below are encoded by `MessageCodec` in the order of the declared parameters in each record. Collection lengths and string/byte lengths are all big-endian int32; list items have no extra tag and are inlined in the declared order for each value type. `GroupToken` is nullable, encoded as a boolean present followed by group, member, and generation; `Acks` is int16, with `LEADER=1` and `ALL=-1`.

API	Request record and fields	Successful-response record and fields
METADATA	MetadataRequest: topic	MetadataBody: partitions

API	Request record and fields	Successful-response record and fields
PRODUCE	ProduceRequest: tp, epoch, acks, timeoutMillis, records	ProduceBody: result
IDEMPOTENT_PRODUCE	IdempotentProduceRequest: tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records	ProduceBody: result
FETCH	FetchRequest: tp, epoch, offset, maxRecords, maxBytes, token	FetchBody: records, logStartOffset, logEndOffset, highWatermark, epoch
COMMIT_OFFSET	CommitOffsetRequest: key, nextOffset, token	EmptyBody
FETCH_OFFSET	FetchOffsetRequest: key	OffsetBody: nextOffset (OptionalLong)
JOIN_GROUP	JoinGroupRequest: group, member, topic	GroupBody: assignment
HEARTBEAT	HeartbeatRequest: token	EmptyBody
LEAVE_GROUP	LeaveGroupRequest: group, member	GroupBody: assignment
GROUP_ASSIGNMENT	GroupAssignmentRequest: group, member	GroupBody: assignment
REPLICA_FETCH	ReplicaFetchRequest: tp, brokerId, epoch, fetchOffset, maxRecords, maxBytes, recoveryRead	ReplicaFetchBody: records, epoch, highWatermark, leaderLogEndOffset

Record	Fields in declaration order
TopicPartition	topic, partition
RecordData	key, value, timestamp, nullable producerStamp
LogRecord	offset, data
AppendResult	firstOffset, nextOffset
OffsetKey	group, tp
Endpoint	host, port
GroupAssignment	generation, assignments
PartitionMetadata	tp, leaderId, epoch, replicas, isr, leader
ProducerStamp	producerId, producerEpoch, firstSequence, batchSize, batchIndex, batchHash
BusinessEvent	eventId, account, delta

Each byte array is length-prefixed; only key may be null (length -1), while value is always present but may be empty.

OffsetStore entries use `MessageCodec.encodeOffsetEntry`, with null key and timestamp 0. The record's physical log offset is independent of committed nextOffset. Business-event values use `MessageCodec.encodeBusinessEvent: version:uint8=1|eventId:identifier|account:identifier|delta:int64`. Both identifiers are non-empty strict UTF-8 strings of at most 255 bytes; delta may be negative, zero, or positive.

A.1.4 Identifiers and Value Types

topic, group, and member are non-empty UTF-8 identifiers with a maximum length of 255 bytes; topic may contain only ASCII letters, digits, periods, underscores, and hyphens, and . and .. are rejected. TopicPartition's partition is nonnegative. RecordData's key may be null, value may not be null (but may be an empty array), and timestamp is nonnegative; LogRecord offset is nonnegative. Once array ownership is transferred to a value object, the receiver owns it, so it need not be copied repeatedly. All offsets that denote the next item use half-open boundaries; see the relevant chapters for the details.

A.2 Idempotent Append and Durable Effects

API id 11 is the separate `IDEMPOTENT_PRODUCE` request; ordinary `PRODUCE` remains non-idempotent and rejects stamped client input. The request carries `tp, epoch, acks, timeoutMillis, producerId, producerEpoch, firstSequence, records`; the broker stamps each record with the request's batch identity and a streamed SHA-256 fingerprint of the record count and timestamp/key/value contents. The caller manages producer id and epoch. A new producer epoch starts at sequence zero and takes effect only when a record is persisted. An exact completed duplicate returns its original offset range without appending; same identity with different content is a conflict; sequence gaps and unknown old sequences are rejected. A matching partial batch can be completed by appending only its missing suffix. Derived state is recovered from retained log history; when `logStartOffset` is nonzero, idempotent requests fail with `PRODUCER_STATE_EXPIRED`. There is no producer-state snapshot or client session WAL.

`IdempotentProducer` sends once per call. A failed call retains its pending batch; only the caller's explicit `retry` refreshes metadata and sends that same identity, sequence, and payload once. Duplicate requests still pass the normal ACK precheck and wait, including ALL's ISR/H-W/deadline rules. This does not change `SimpleProducer`, which never retries automatically.

`DurableEffectStore` applies one `BusinessEvent` as one forced local log record. The first event id appends and updates its account balance; an identical id/content replay returns false without another append; an id reused with different content or a checked-balance overflow is rejected. Recovery rebuilds event identities and balances from the retained log and requires log start zero. This local ledger is not atomically coordinated with `ProcessingLoop`'s offset commit: the callback may run again after a commit failure, while a stable event id prevents a duplicate local accounting effect. Group fencing cannot undo an effect that already happened.

A.3 Fixed Error Codes

Wire ID	Name	Purpose
0	NONE	Success.
1	INVALID_REQUEST	Invalid parameter, length, version, or request format.
2	UNKNOWN_TOPIC_OR_PARTITION	Topic/partition does not exist.
3	OFFSET_OUT_OF_RANGE	Offset violates the current log start/end constraints.
4	CORRUPT_RECORD	Complete-record CRC/format error or conflict with committed history.
5	NOT_LEADER	The requester is not the current leader.
6	FENCED_EPOCH	The request uses an old leader epoch.
7	NOT_ENOUGH_REPLICAS	The current ISR has fewer than minISR members.
8	REQUEST_TIMEOUT	The request deadline expired; the outcome may be unknown.
9	ILLEGAL_GENERATION	The consumer-group generation has expired.
10	UNKNOWN_MEMBER	The consumer-group member is not registered.
11	NOT_ASSIGNED	The operation involves an unassigned partition.
12	NO_ELIGIBLE_LEADER	No candidate satisfies the clean-election conditions.
13	STORAGE_ERROR	I/O failure; the underlying cause is retained.
14	FENCED_PRODUCER_EPOCH	The request uses an older producer epoch.
15	OUT_OF_ORDER_SEQUENCE	The batch sequence or boundary is invalid.
16	DUPLICATE_SEQUENCE_CONFLICT	A repeated batch identity has different content.

Wire ID	Name	Purpose
17	PRODUCER_STATE_EXPIRED	Required producer history is outside the retained log prefix.
18	INCOMPLETE_BATCH	An unrelated append would cross an incomplete producer batch.

An unknown ID is not mapped to success; I/O errors do not swallow the cause; network-error bodies do not return the server stack trace.

A.4 The 31 Steps and Edit Entry Points

All paths are relative to the repository root. Students modify the specified methods under `exercises/src/main/java/io/simplekafka/`. The cumulative command for each step is `./gradlew :test -Pstep=N`; the single-step diagnostic is `./gradlew :stepTest -Pstep=N`, which does not prove progression.

Step	Course title	Edit methods
1	Record encoding and decoding	storage/RecordCodec: encode, decode
2	Log append	storage/SegmentLog: append
3	Offset-based reads	storage/SegmentLog: readFrom
4	Crash recovery	storage/SegmentLog: recover
5	Sparse index	storage/SparseIndex: add, floor, rebuild
6	Segmented log	storage/PartitionLog: append, read
7	Log retention	storage/PartitionLog: deleteBefore
8	Tail truncation	storage/PartitionLog: truncateTo
9	Partition routing	client/Partitioner: choose
10	Topic metadata	broker/PartitionCatalog: createTopic, metadata
11	TCP framing	protocol/FrameCodec: write, read
12	Single-broker requests	broker/BrokerHandler: handle
13	Batched producer	client/SimpleProducer: send, flush
14	Pull-based consumer	client/SimpleConsumer: assign, seek, poll
15	Consumer persistence	group/OffsetStore: commit, fetch; client/SimpleConsumer: resume, commitSync; broker/BrokerHandler: COMMIT_OFFSET, FETCH_OFFSET
16	At-least-once processing	client/ProcessingLoop: runOnce
17	Partition assignment	group/RoundRobinAssignor: assign
18	Membership and rebalancing	group/GroupCoordinator: join, leave, assignment; broker/BrokerHandler: JOIN_GROUP, LEAVE_GROUP, GROUP_ASSIGNMENT
19	Heartbeats and expiration	group/GroupCoordinator: heartbeat, expire; broker/BrokerHandler: HEARTBEAT
20	Consumer fencing	group/GroupCoordinator: validate; client/GroupConsumer: subscribe, poll, commitSync; broker/BrokerHandler: FETCH/COMMIT_OFFSET token fencing

Step	Course title	Edit methods
21	Fetch-based replication	broker/BrokerHandler: handle(REPLICA_FETCH); replication/FollowerReplicator: pollOnce
22	ISR and high watermark	replication/ReplicationTracker: report, expireLagging, advanceHighWatermark
23	Produce acknowledgments	replication/AckPolicy: validateBeforeAppend, await
24	Concurrent replicated broker	replication/ReplicatedPartition: produce, fetch
25	Clean election and leader epochs	replication/LeaderElection: elect, checkLeader
26	Divergent log reconciliation	replication/ReplicaReconciler: reconcile
27	Metadata routing	client/MetadataRouter: refresh, call refresh
28	Replica admission to ISR	replication/ReplicaAdmission: tryAdd
29	Cross-layer consistency experiments	lab/ConsistencyExperiments: producerOutcomeUnknown, leaderAckLoss, allAckTimeout, consumerReplay, staleGroupSideEffect
30	Idempotent producer	client/IdempotentProducer: send, retry; replication/IdempotentAppender: append, recover; replication/ReplicatedPartition: produceIdempotent; broker/BrokerHandler: handle(IDEMPOTENT_PRODUCE); client/MetadataRouter: call; storage/RecordCodec: encode, decode; storage/SegmentLog: append, recover; storage/PartitionLog: append, deleteBefore, truncateTo
31	Idempotent consumer effects	client/DurableEffectStore: apply, recover

The first seven chapters cover four steps each; Chapter 8 covers Steps 29–31. The cumulative pass points at chapter ends are 4, 8, 12, 16, 20, 24, 28, and 31. If an earlier step has not passed, a later test will usually fail as well, because the course develops one evolving system rather than unrelated demo programs.

A.5 Course Simplifications and Differences from Kafka

1. **Routing:** keys use CRC32C unsigned modulo; Kafka’s default key partitioning algorithm is different. null keys use per-instance round robin rather than Kafka’s sticky batching.
2. **Disk:** records are encoded individually and each batch explicitly calls `force(false)`; this is not Kafka record batches or its default page-cache path. Forcing a write does not guarantee media-level power-loss safety.
3. **Index:** write one sparse point per fixed number of records, not at byte intervals.
4. **Consumer groups:** a simplified model with one topic and immediate rebalancing, not the full classic barrier, cooperative rebalancing, or KIP-848. Token fencing protects offset commits inside the course broker only; it does not make external side effects atomic.
5. **Offsets:** the local OffsetStore is not the highly available, replicated/compacted `__consumer_offsets`.
6. **Replication and control plane:** each broker fetches over real TCP and writes to an independent directory, but the authority/coordinator is single and trusted in the same JVM.

This is not KRaft and does not guarantee cluster recovery after controller/authority failure or consensus under arbitrary partitions.

7. **Visibility assumption:** the course retains the committed prefix and requires a clean candidate's LEO to be no lower than the authority HW, so the known commit boundary does not move backward during a controlled failover. This is stronger than real Kafka's HW behavior across epochs and must not be generalized into a claim that Kafka is always monotonic.
8. **Retries, idempotence, and transactions:** the ordinary `SimpleProducer` never retries automatically and a timeout may follow an append. The separate `IdempotentProducer` supports caller-managed producer identity and explicit retry, using batch stamps persisted in the log; it does not implement Kafka PID allocation, a transaction coordinator, client session WAL, or producer-state snapshots, and retention can expire the required prefix. The local `DurableEffectStore` deduplicates by stable business event id but is not atomic with consumer-offset commits. There are no Kafka transactions or `read_committed` guarantees, compaction, fire-and-forget, or ELR. At-least-once allows duplicates; this teaching subset does not claim end-to-end exactly-once.
9. **Recovery:** Step 26's full-prefix comparison is an $O(n)$ teaching simplification; real systems use leader-epoch history to narrow the divergent-log search range.

A.6 Official References and Further Reading

The references below explain Kafka concepts and differences in real systems; they are not specifications for the course's custom wire/storage layouts. Version labels reflect the pages cited by this book; configuration and protocol details can change between versions, so consult each page at its listed version.

Topic	Official reference	Related chapters
Kafka design and persistence model	Kafka 4.2 Design	Official overview of logs, replication, and consumer positions; see the simplifications above for this course's differences.
Consumer positions and commits	KafkaConsumer 4.2 API	Chapters 14–16: position, seek, commit, and thread constraints.
Minimum ISR configuration	Kafka 4.3 Topic Configs	Chapters 22–24: <code>min.insync.replicas</code> and write acknowledgments.
Consumer-group protocol differences	Kafka 4.3 Rebalance Protocol	Chapters 17–20: context for the classic and newer protocols; the course does not implement the full protocol.
Leader epochs and truncation	KIP-101	Chapter 26: why HW alone is insufficient to locate a divergent log.
JUnit test execution	JUnit 5.11.4 User Guide	Read actual Step behavior failures and test reports.
Tectonic compile options	Tectonic compile	The book uses XeTeX/CJK with the declared bundle and local cache; see the build notes below.

A.7 Typesetting and Build Instructions

The English entry point uses `book[openany,oneside,11pt]`; the Chinese entry point uses `ctexbook[UTF8,fontset=none,openany,oneside,11pt]`. Both use the shared preamble and then load their language-specific strings. `fontspec` and `xeCJK` select fonts by file from the TeX bundle rather than scanning system fonts. Latin Modern roman uses `lmroman10-regul`

`ar.otf` and its matching bold/italic/bolditalic files; sans uses `lmsans10-regular.otf` and its bold/oblique/boldoblique files; mono uses `lmmono10-regular.otf` and `lmmono10-italic.otf`, with bold using the regular file and bolditalic using the italic file. CJK serif uses `FandolSong-Regular.otf` with `FandolSong-Bold.otf`, and `FandolKai-Regular.otf` for italic and bold italic; CJK sans uses `FandolHei-Regular.otf` and `FandolHei-Bold.otf`; CJK mono uses `FandolFang-Regular.otf` for every shape. The build does not depend on Noto, fontconfig, xelatex, or latexmk. Code snippets use listings and show only ASCII Java identifiers and short interfaces; Chinese explanations are written in the text. The book does not depend on minted, shell escape, or embedded web pages. TikZ diagrams are drawn locally by TeX. EPUB conversion uses Pandoc 3.12.1 and the same expanded TeX sources; Tectonic renders the six TikZ figures, and Poppler's `pdftoppm` rasterizes them for embedding.

`./gradlew :setup` resolves the verified JUnit dependency; `./gradlew :setupBook` prepares checksum-verified Tectonic 0.17.0 and Pandoc 3.12.1 under `.tools/`, and checks for Poppler's `pdftoppm`. Poppler must be installed separately; `:setupBook` does not compile Java or resolve JUnit, and the Wrapper needs Java 17+ to launch it. `./gradlew :book` builds the English edition by default with `COURSE_LANG=en`; `COURSE_LANG=zh` `./gradlew :book` builds the Chinese edition. The English PDF/EPUB are written to `build/book/en/simpleKafka.pdf` and `build/book/en/simpleKafka.epub`; the Chinese pair is written to `build/book/zh/simpleKafka-zh.pdf` and `build/book/zh/simpleKafka-zh.epub`. Each command updates only its language's checked-in files under `docs/book/dist/`; readers can open those files without Java or build tools. TECTONIC can override the default repository tool, PANDOC must resolve to version 3.12.1, and TEX_BUNDLE can specify a bundle URL or local directory/zip/ttb. The cache is stored in `.tools/tectonic-cache`. The first build needs the online bundle, which supplies TeX packages and fonts. Build both language editions online first to warm the resources; afterward, `BOOK_OFFLINE=1` uses Tectonic's only-cached mode for PDF and diagram builds, while Gradle's `--offline` separately controls Gradle dependency resolution. The book tasks do not compile Java or use JUnit. The generated PDF and EPUB files are not promised byte-for-byte reproducible. The repository-pinned Tectonic engine is version 0.17.0; Pandoc is version 3.12.1.

Glossary

Term	Meaning and location in this book
Append	Write records at the log tail using consecutive offsets; Chapters 1, 2, and 6.
Offset	A logical record number in a partition log, not a file byte position; Chapter 1.
Byte position	The physical starting position of a record in one segment file; Chapters 1 and 2.
LEO	Log End Offset, the offset of the next record that can be written; Chapters 1 and 6.
HW	High Watermark, the committed boundary visible to ordinary consumers; Chapters 6 and 7.
ISR	In-Sync Replicas, the set of replicas that satisfy this course's synchronization conditions with the current leader; Chapters 6 and 7.
Generation	A version assigned to consumer-group membership, used to reject commits from stale members; Chapter 5.
Leader epoch	A version of the partition leader role; requests from the previous role are fenced after a transition; Chapter 7.
At-least-once	Processing may be repeated, but records that were not successfully processed are not silently skipped before commit; Chapter 4.